

机器学习与深度学习

陈老师

2024-10



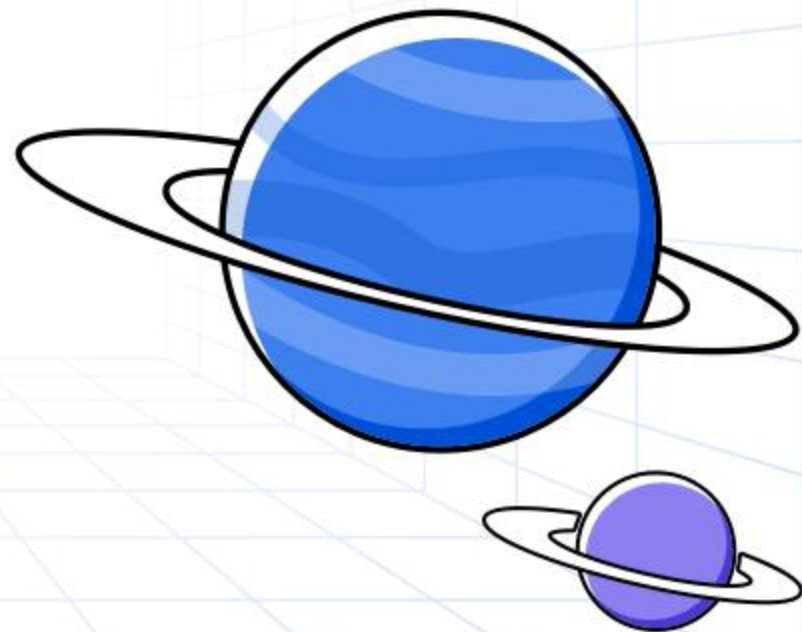
| CATALOGUE |

目录

- 机器学习
- 特征选择
- 常见机器学习方法
- 深度学习
- 算法分析
- 算法可视化分析
- 训练数据收集与算法模型训练
- 神经网络算法效果评价
- 算法检测与分类效果评价

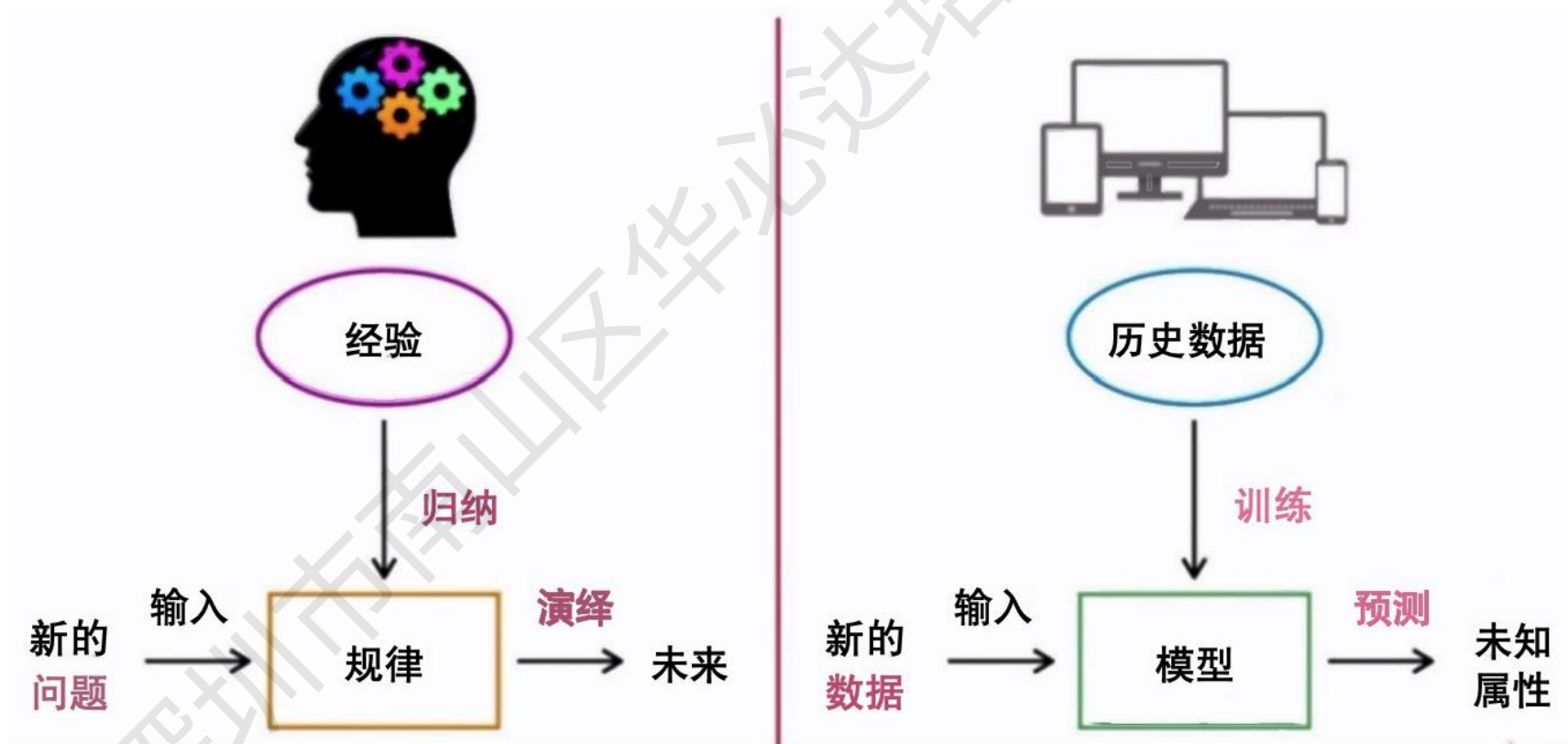
01

机器学习



人类思考和机器学习

人类思考 vs 机器学习



相关概念

- 数据挖掘 (**Data mining**)

从大量数据中提取隐含的、事先未知且潜在有用的知识的过程。

从数据中提取隐藏模式的过程。

- 机器学习 (**Machine learning**)

通过计算的手段，利用经验来改善系统自身性能的过程。

主要研究在计算机上从数据中产生模型的算法。

- 人工智能 (**Artificial intelligence**)

使机器做那些由人需要通过智能来做的事情的科学。

关于知识的科学（知识的表示、获取和运用）。

科学发现的“第四范式” ——

数据密集型科学发现：处理来自技术、社会和经济系统的大量数据，以发现一些隐藏的模式。

“The fourth paradigm”

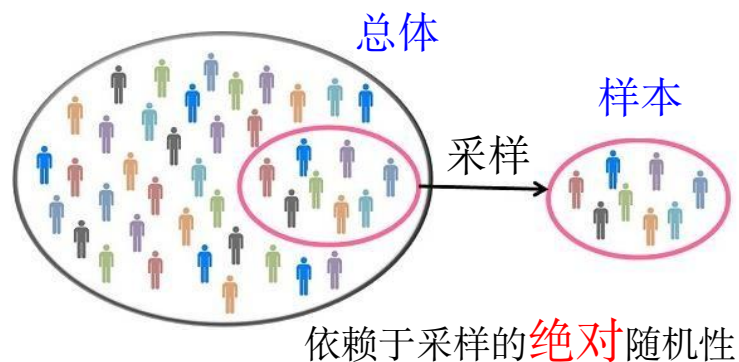


Research purpose lies in handling huge amounts of data from technological, sociological, and economic systems to discover some hidden patterns.

Jim Gray

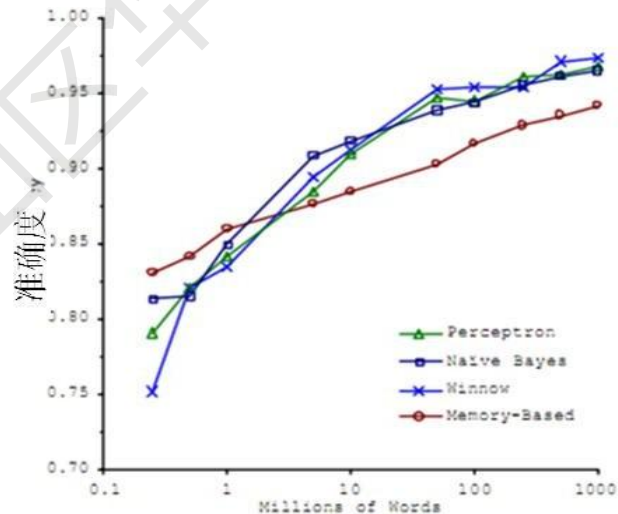
大数据

■ 所谓“大”—— 样本即总体



不用随机分析法（抽样调查）这样捷径，而采用所有数据进行分析处理。

—— 舍恩伯格《大数据时代》，2013

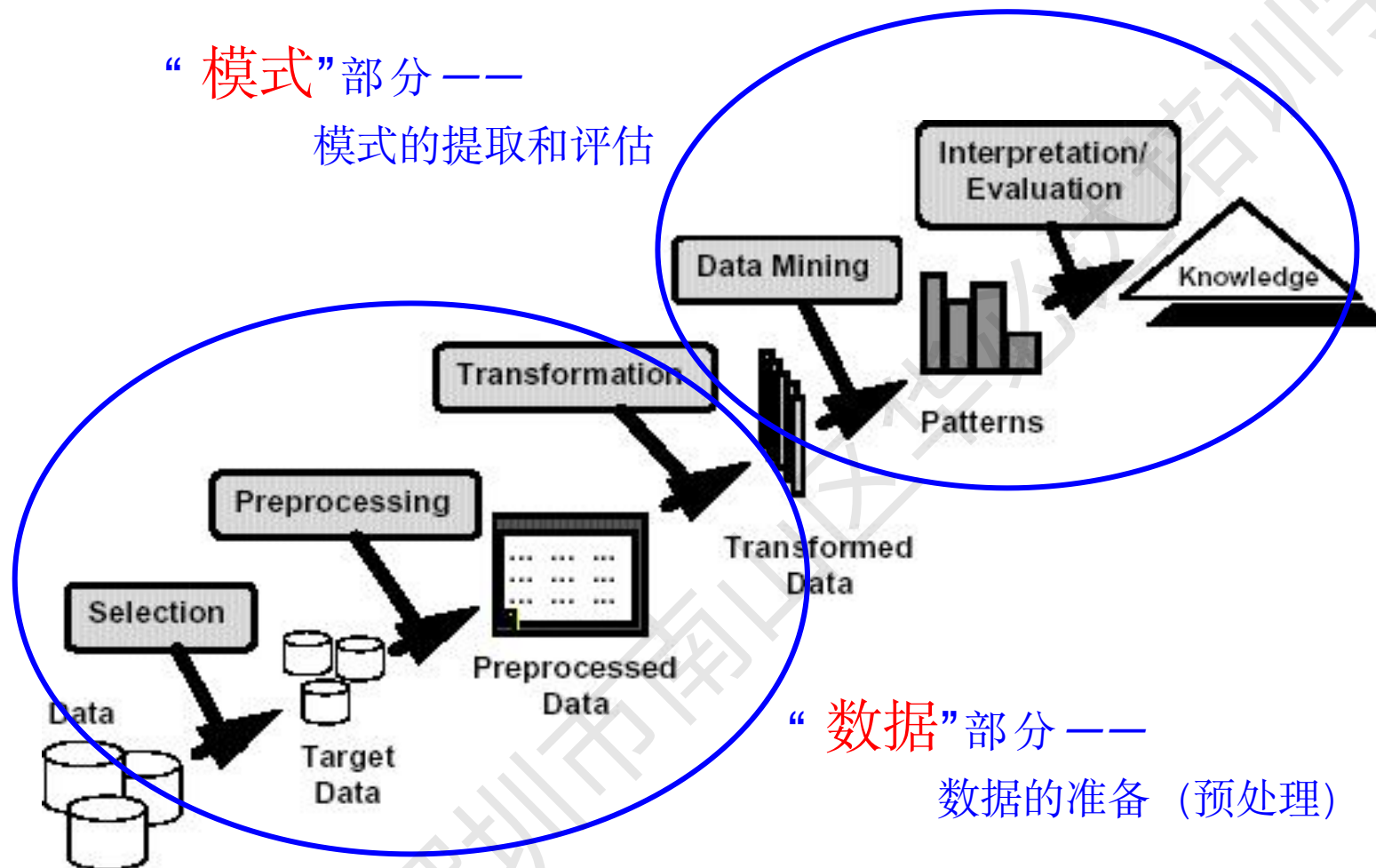


“It’s not who has the best algorithm that wins.
It’s who has the most data.”

一般过程（Knowledge Discovery in Database, KDD）

“模式”部分——

模式的提取和评估



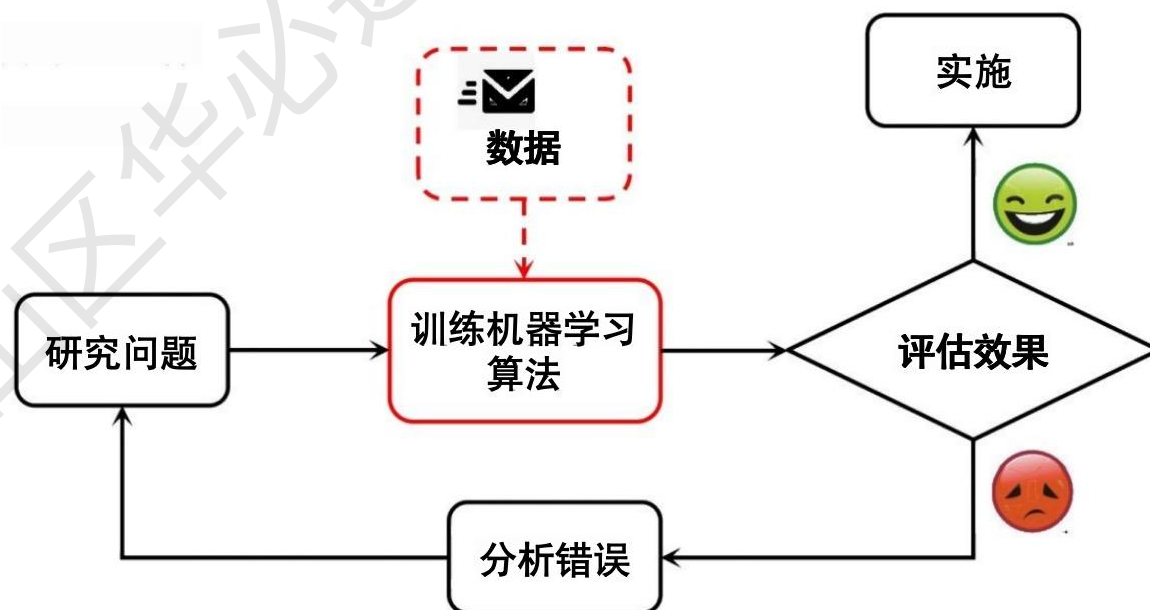
“数据”部分——

数据的准备（预处理）

机器学习方法

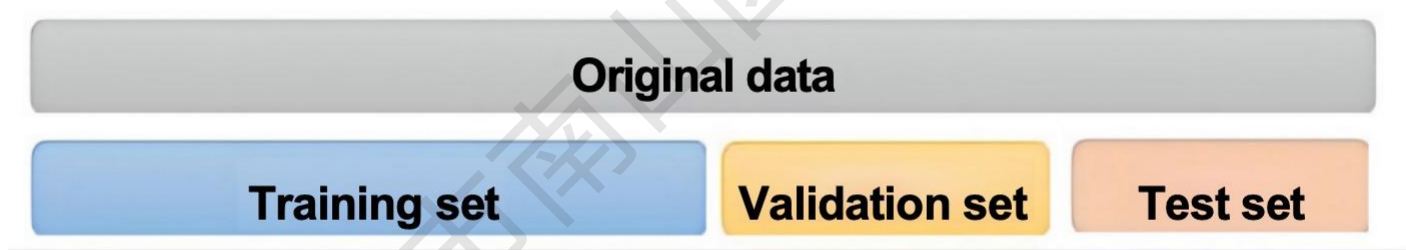
> 机器学习方法

- 以数据为核心
- 在数据中自动学习
 - 自动发现规律
 - 自动适应变化



划分数据集

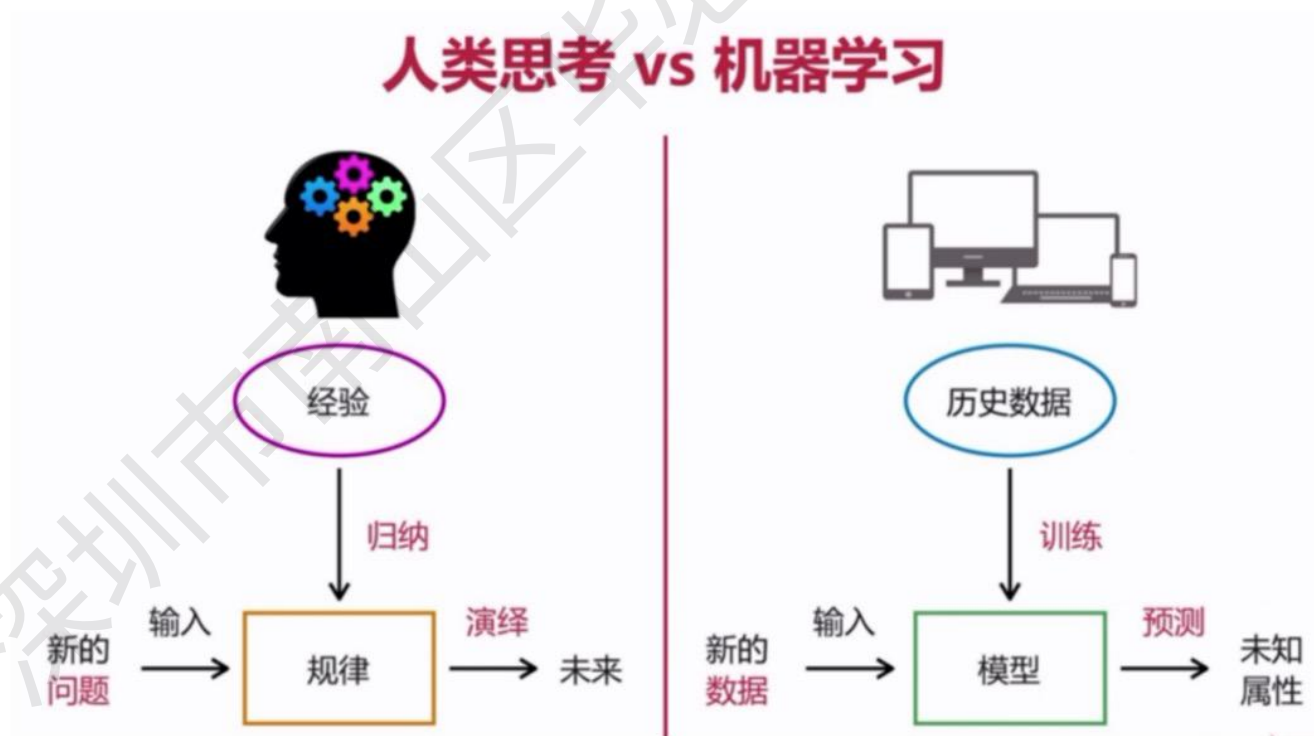
- > 在**训练集**上训练
- > 在**验证集**上调整模型
- > 在**测试集**上测试



所谓机器学习

➤从数据中学习

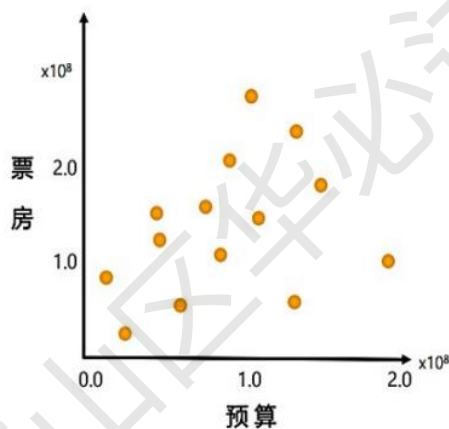
- 学什么？
 - 建立输入和输出之间的关系
- 怎么建立？
 - 函数



电影预算与票房

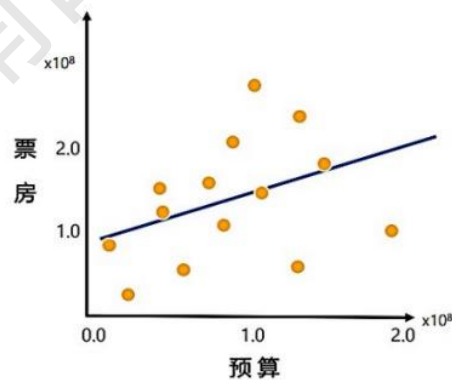
➤关注电影预算和票房之间的关系

- 自变量
 - 电影预算
- 因变量
 - 电影票房
- 用一条**直线**来拟合分布规律
 - $\hat{y} = \theta_0 + \theta_1 x$
 - 注意，也可以用曲线拟合
 - 函数也跟着变



$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

Annotations:
- $y_{\beta}(x)$ points to '票房收入' (Box Office Revenue)
- β_0 points to '系数 0' (Coefficient 0)
- $\beta_1 x$ points to '系数 1' (Coefficient 1) and '电影预算' (Movie Budget)



$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

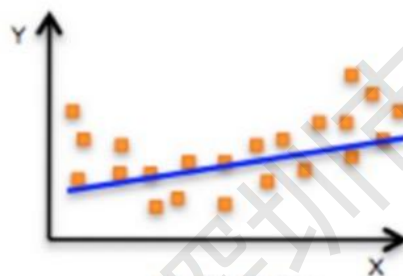
$\beta_0 = 8000 \text{ 万}, \beta_1 = 0.6$

拟合

➤ 已知一个点集（数据集）

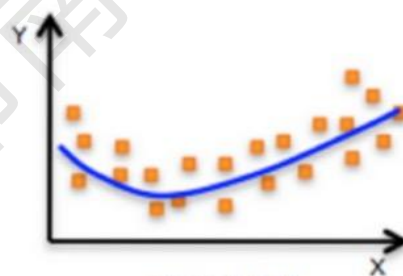
- 构建函数

- 如 $\hat{y} = \theta_0 + \theta_1 x$
- θ_0 和 θ_1 都叫参数
- θ_0 又叫偏置/偏差
- θ_1 又叫权重



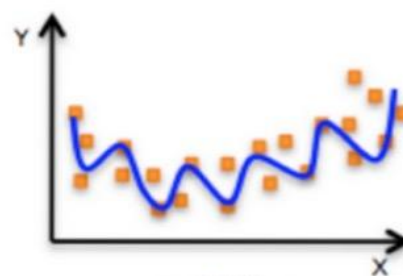
Underfitting

$$\theta_0 + \theta_1 x$$



Just right!

$$\theta_0 + \theta_1 x + \theta_2 x^2$$



overfitting

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

拟合

➤ 已知一个点集（数据集）

- 构建函数

- 如 $\hat{y} = \theta_0 + \theta_1 x$

- θ_0 和 θ_1 都叫参数

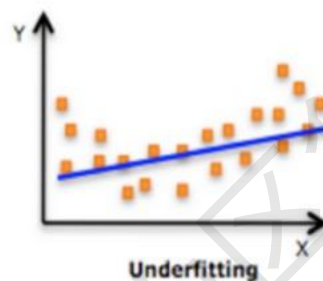
- θ_0 又叫偏置/偏差

- θ_1 又叫权重

- 调整函数中的参数

- 使得该函数与已知点集的差别最小

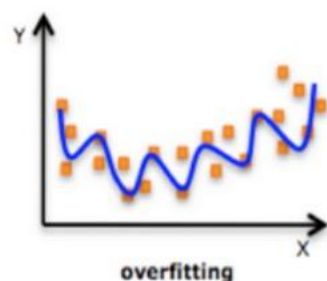
- 预测值和真实值的差别最小



$$\theta_0 + \theta_1 x$$



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

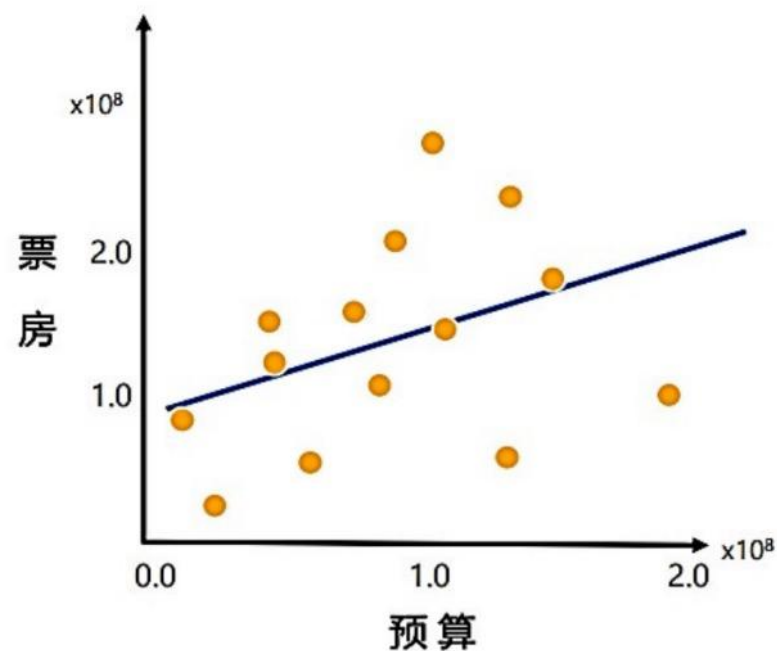
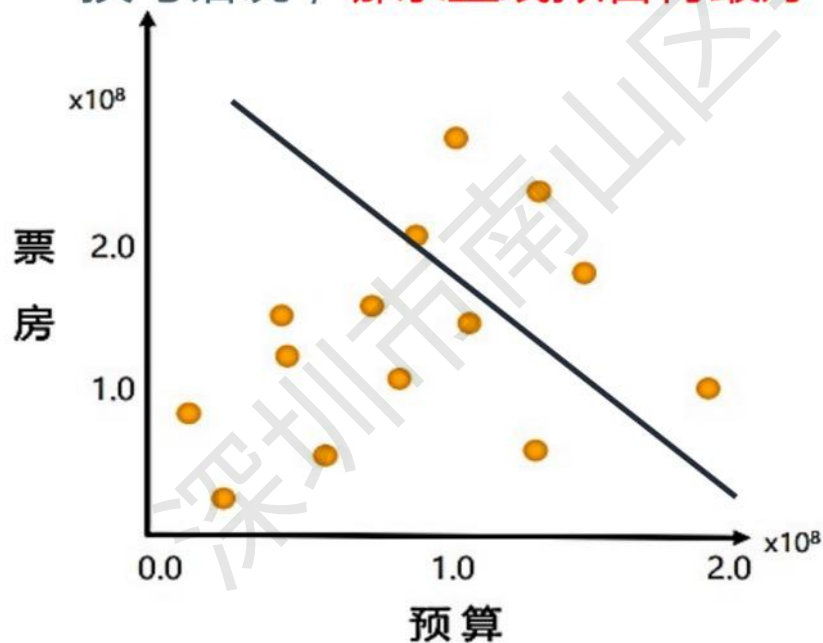


$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

电影预算与票房

➤ 在这些点中有多条备选的直线

- $\hat{y} = \theta_0 + \theta_1 x_1$
 - 不同的 θ_0 和 θ_1 的取值
 - 如何寻找最优的参数向量 θ ？
 - 换句话说，哪条直线拟合得最好？

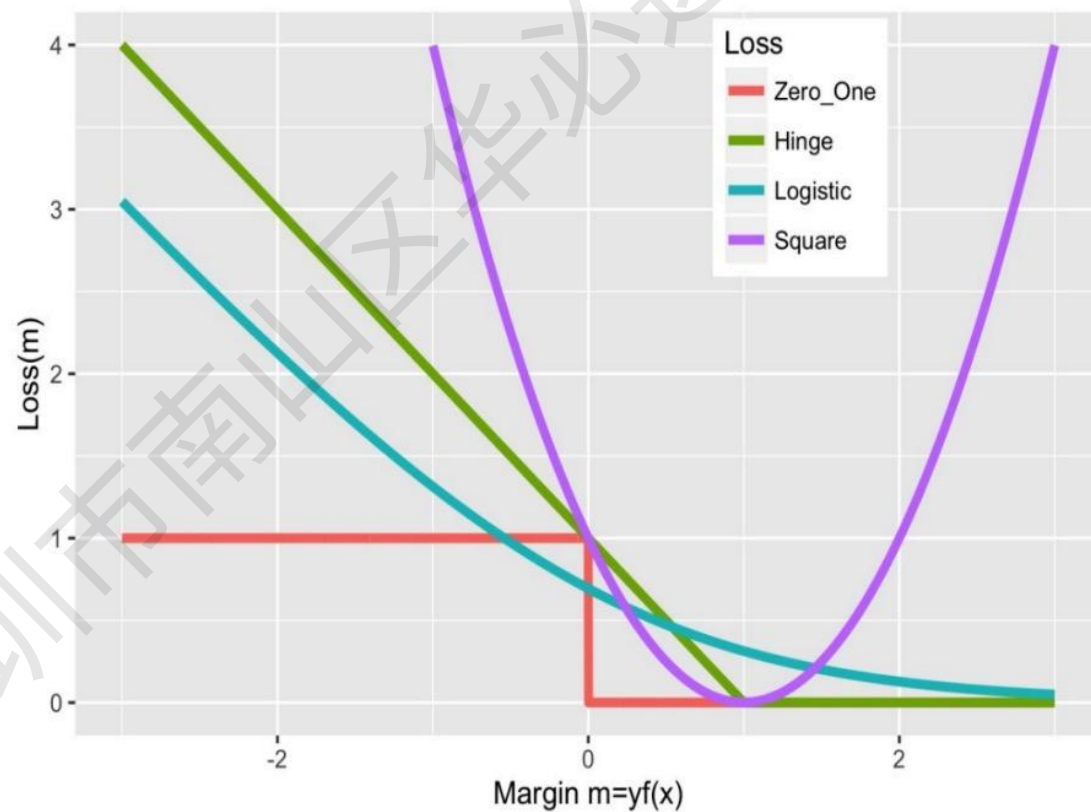


哪条直线拟合得最好？

➤模型的预测值越接近真实值一般来说越好

- 我怎么知道预测值和真实值差多少？

- 使用损失函数



什么是损失函数？

➤ 损失函数

- 衡量被拟合数据（真实值）与模型预测（预测值）之间差距的函数

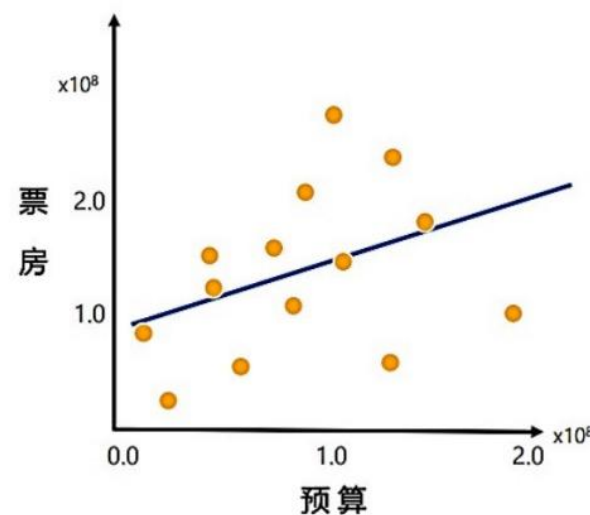
- 比如均方误差

- Mean Square Error , MSE

- $MSE(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T \cdot \mathbf{x}^{(i)} - y^{(i)})^2$

预测值

真实值



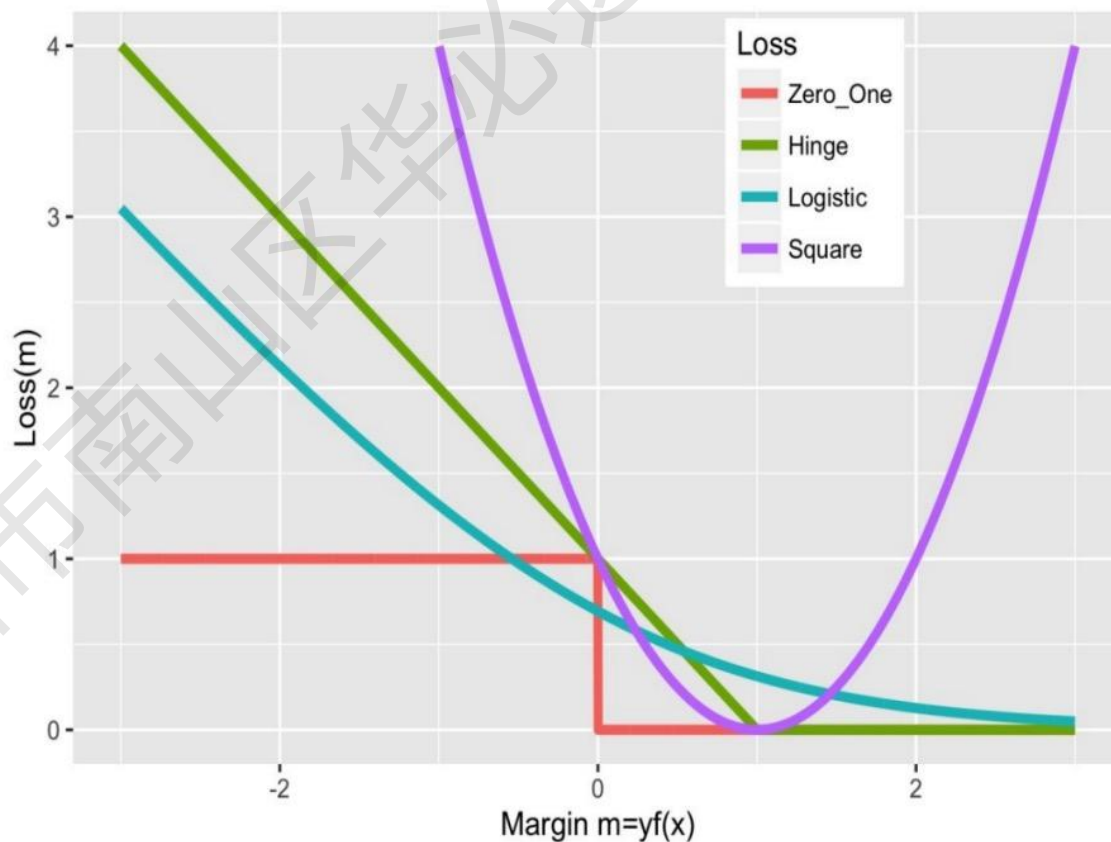
预测值和真实值差多少？

➤ 换句话说，我怎么知道拟合得怎么样？

- 转化为最优化问题

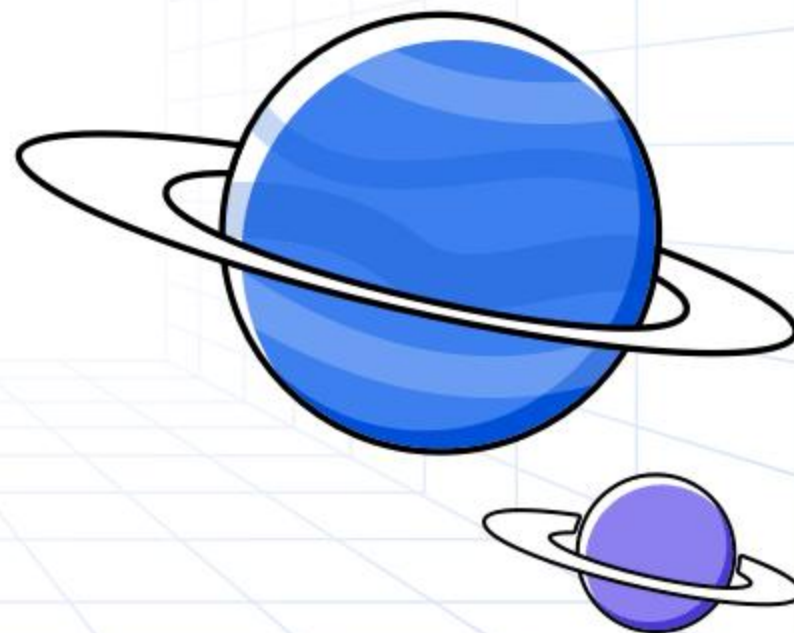
- 最小化损失函数

- 如何做？



02

特征选择



庞大特征数量带来的困扰

➤通常机器学习中特征数量很多

- 例如文本处理时
 - 将一个词作为一个特征
 - 特征数量可能达到上万个
- 可能存在特征组合
- 可能很稀疏
 - 有一个含有100万单词的词典
 - 一句话中特定单词的出现次数



庞大特征数量带来的困扰

➤特征过多显然会带来问题

- 最好是使无意义维度的权重降至0
 - 正好为0的权重基本上会使相应特征从模型中移除
 - 将特征设为0可节省存储空间，且可以减少模型中的噪点
- 我们希望尽可能降低模型的复杂度
 - 奥卡姆剃刀定率



添加模型复杂度惩罚项

➤ 衡量模型复杂度有多种方式

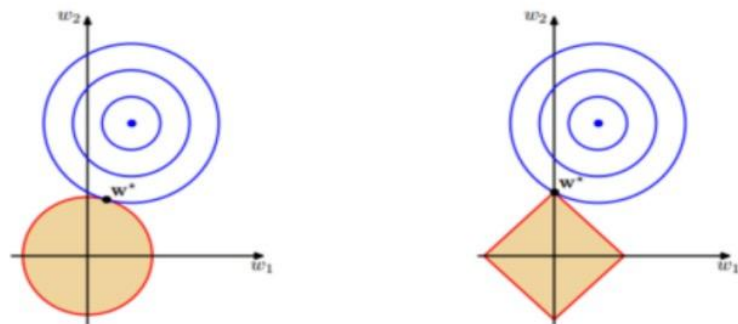
- 将模型复杂度作为模型中所有特征的权重的函数
- 将模型复杂度作为具有非零权重的特征总数的函数
-
- 如果模型复杂度是权重的函数
 - 则特征权重的绝对值越高，对模型复杂度的贡献就越大



添加模型复杂度惩罚项

➤ 将这种想法纳入训练时所进行的优化中

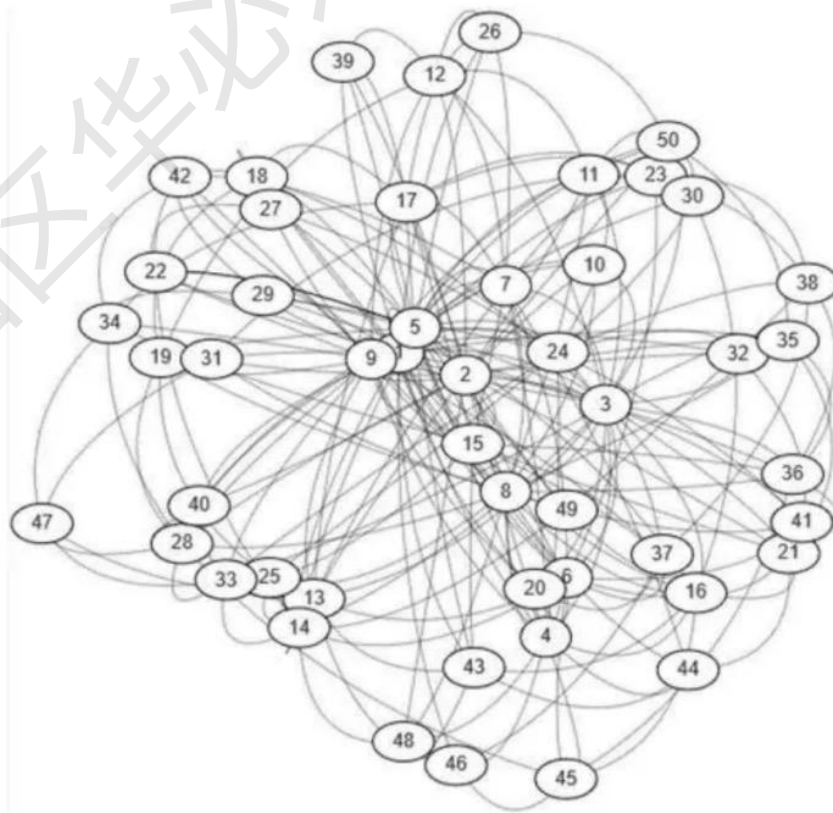
- 训练专注于获取正确的样本
 - 最大程度减少经验风险（经验风险最小化）
 - minimize: $\text{Loss}(\text{Data} \mid \text{Model})$
- 引入第二项因素
 - 即最小化损失，但同时平衡模型复杂度（结构风险最小化）
 - minimize: $\text{Loss}(\text{Data} \mid \text{Model}) + \text{complexity}(\text{Model})$



添加模型复杂度惩罚项

➤ 训练优化算法是一个由两项内容组成的函数

- 一个是**损失项**
 - 用于衡量模型与数据的拟合度
- 另一个是**正则化项**
 - 用于衡量模型复杂度



特征选择方法



过滤法

基于统计性质进行特征选择，如方差分析、相关系数等。

包裹法

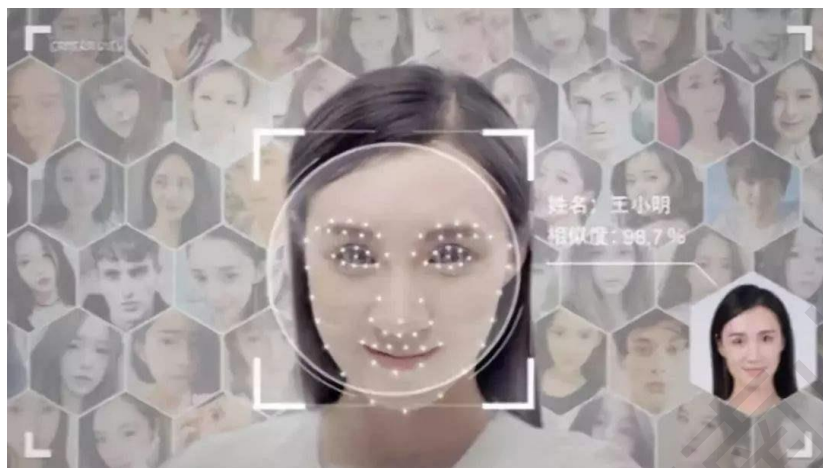
根据模型性能进行特征选择，如递归特征消除。

嵌入法

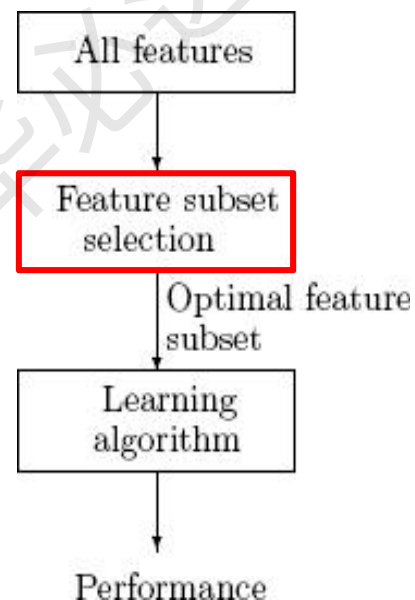
在模型训练过程中进行特征选择，如L1正则化、决策树等。

过滤法

Filter: 作为数据挖掘一个预处理步骤，根据“经验”选择特征子集
(因此，其性能的评判标准独立于目标算法)



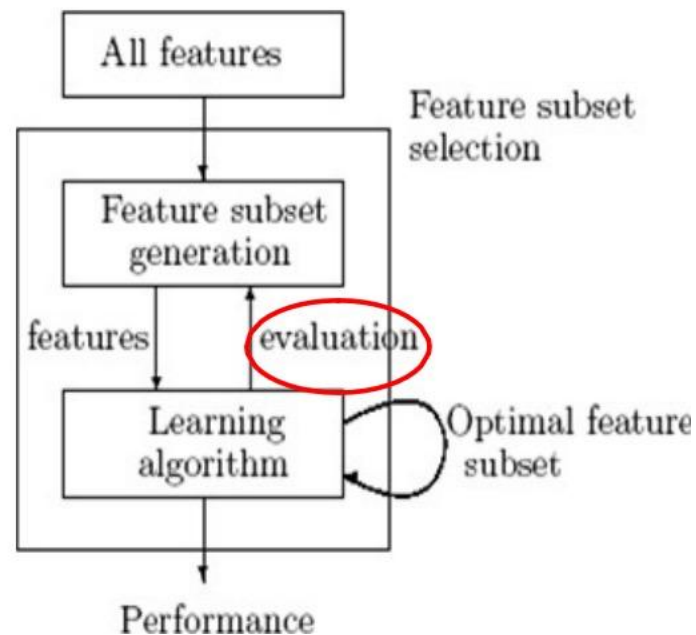
— 一种特征工程 (feature engineering) 方法



封装法

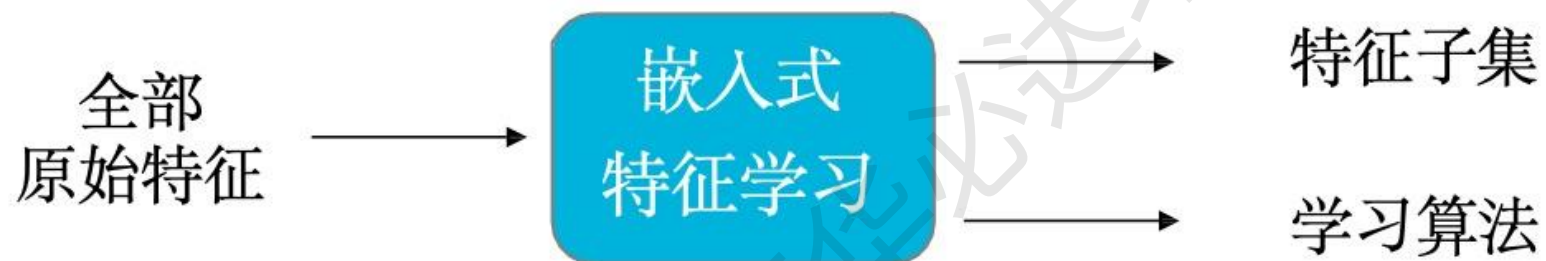
Wrapper：选择一组令目标算法取得最佳性能的特征子集。

- 使用与过滤法相同的候选特征子集生成方法。
- 不同的是，使用目标算法的性能为子集优劣的评价指标
 - 一定可以取得“最好”的性能；
 - 但有过拟合（**over-fitting**）风险



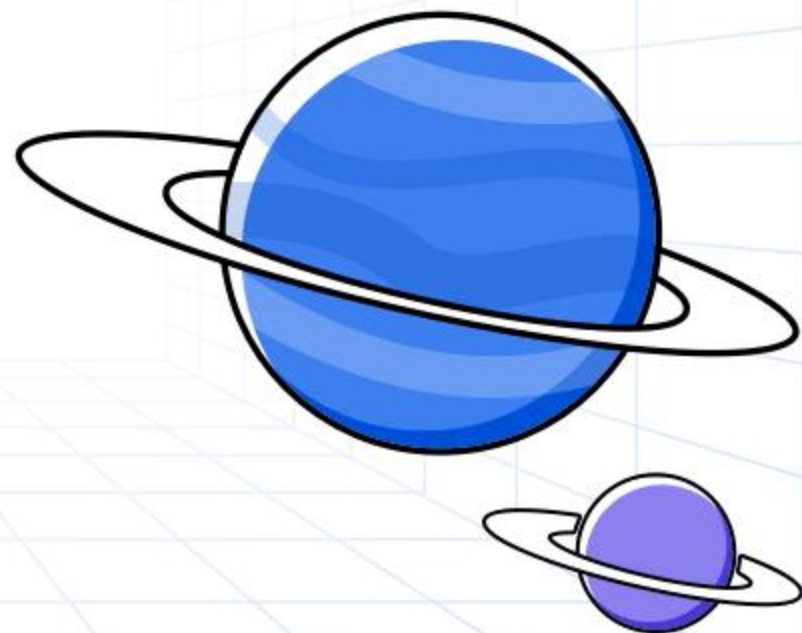
嵌入法

Built-in method: 将特征选择看作是优化目标的一部分



03

常见机器学习方法



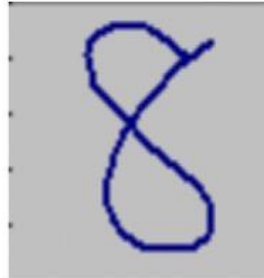
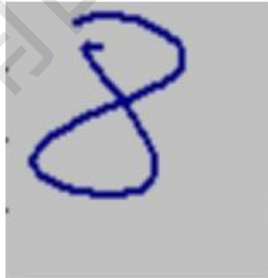
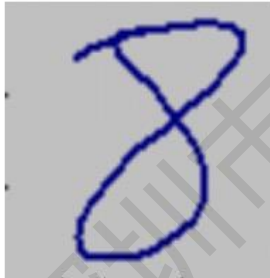
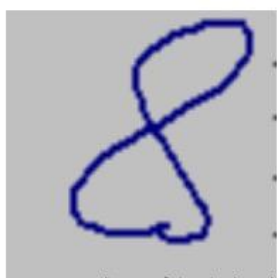
类模式挖掘

■ 物以类聚

- 人类认识自然界的一个重要途径
- 根据经验知识，将具有共同特征的事物归纳为相同的类别



→ “树”



→ “8”

一个类就是一个概念（模式）。

问题是如何判断它们是否能“聚”在一起？相似性计算！

监督学习

■ (有) 监督学习 (Supervised learning)

—— 在“老师”指导下的机器学习



已知：一些花的数据

萼片长度	萼片宽度	...	花的类型
5.1	3.5	...	Setosa
4.9	3.0	...	Setosa
4.7	3.2	...	Versicolor
1.6	5.3	...	Virginica

要求：预测这朵花的类型

萼片长度	萼片宽度	...	花的类型
5.0	3.4	...	?

训练一个模型



预测



— 这些数据称为训练数据

训练数据中已知的类别标签就是“老师”！

监督学习就是从训练数据（“历史”）总结规律，继而去推断“未来”。

无监督学习

■ 无监督学习 (Unsupervised learning)

—— 没有老师指导下的机器学习 对
无标记数据进行的机器学习

要求：这些花有几种类别？每朵花分别属于什么类型？

萼片长度	萼片宽度	...	花的类型
5.1	3.5	...	?
4.9	3.0	...	?
4.7	3.2	...	?
1.6	5.3	...	?

无**训练数据**，因此也无“老师”。

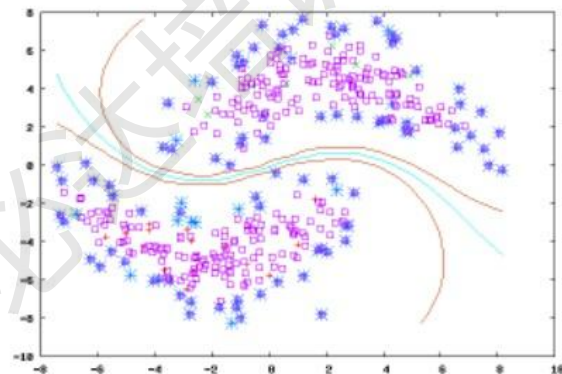
—— 只能根据数据**内在的**统计特性进行类别划分。



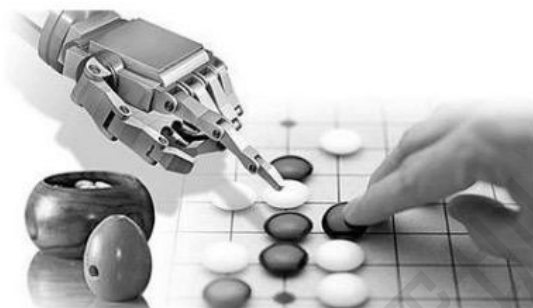
其他学习方法

■ 半监督学习 (Semi-supervised learning)

- 只有少量训练数据;
- 带有约束条件 (背景知识)



■ 增强学习 (Reinforcement learning)



动作

奖励



- 以对手的反馈 (一个动作获得的回报) 为“监督”信息。

机器学习三大问题

分类

回归

聚类



回归

➤ 回归问题

- 模型输出为连续数据
- 有监督学习
- GDP预测，股价预测等



分类

■ Classification

- 给定一组训练样本

- 👉 每个样本用一组属性(attribute)来描述

- 👉 每个样本具有已知的类别(class), 称为类标号

- 构造一个模型 (属性集到类别的映射函数)

$$\text{class} = f(\text{attributes})$$

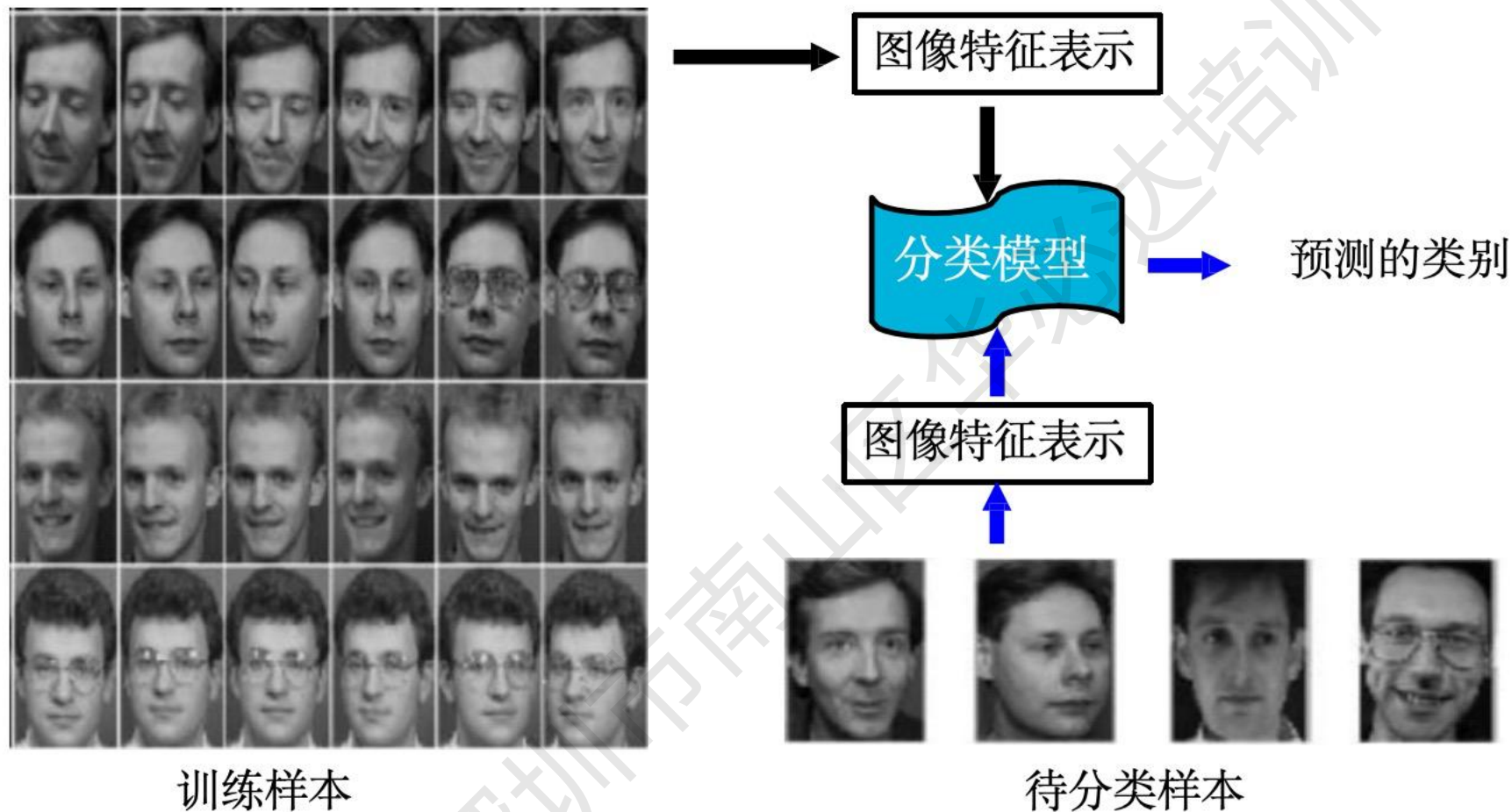
- 目的: 预测未知类别样本 (但其属性new_attribute已知)

$$\text{预测的类别} = f(\text{new_attribute})$$

■ 分类是典型的监督机器学习

分类一般过程

■ 以人脸识别为例



两阶段任务：建立模型 (训练阶段) → 使用模型 (分类阶段)

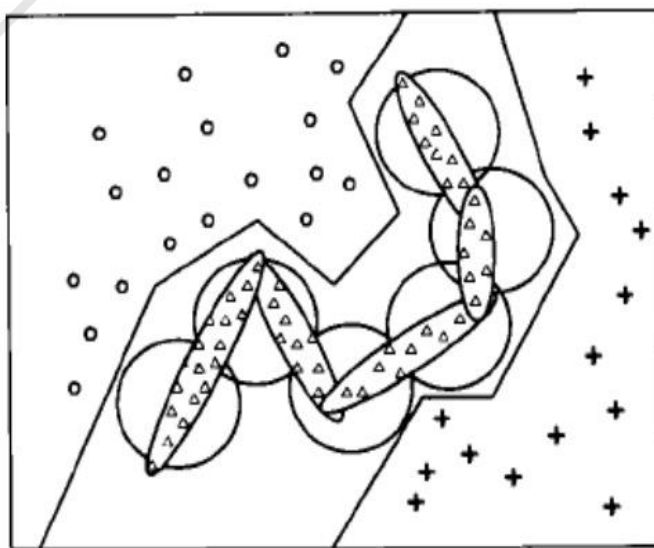
主要方法分类

- 以是否构造了模型为区分标准

- “懒” (Lazy) 分类器: 不显式地建立学习模型
代表作: k -NN算法
- “热” (Eager) 分类器:
代表作: SVM(支撑向量机)、ANN (人工神经网络) ...

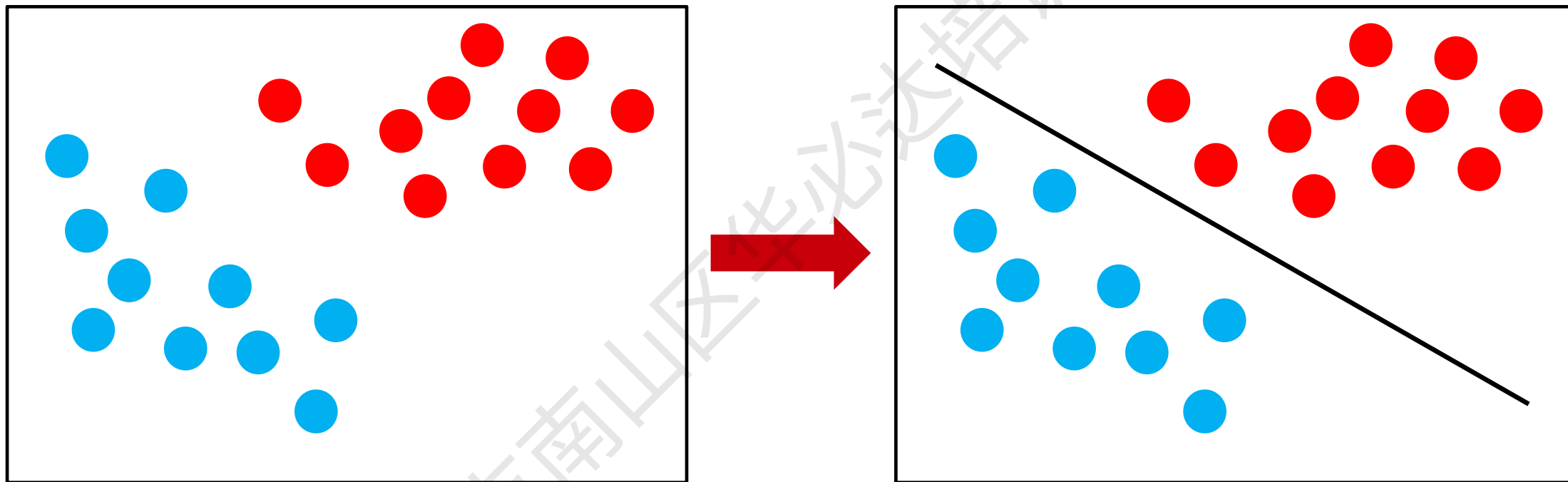
- 以分类模型的表达形式为区分标准

- 以样本在空间中的最佳**划分**为目标的分类
代表作: SVM、ANN
- 以样本在空间中的最佳**覆盖**为目标的分类
代表作: k -NN



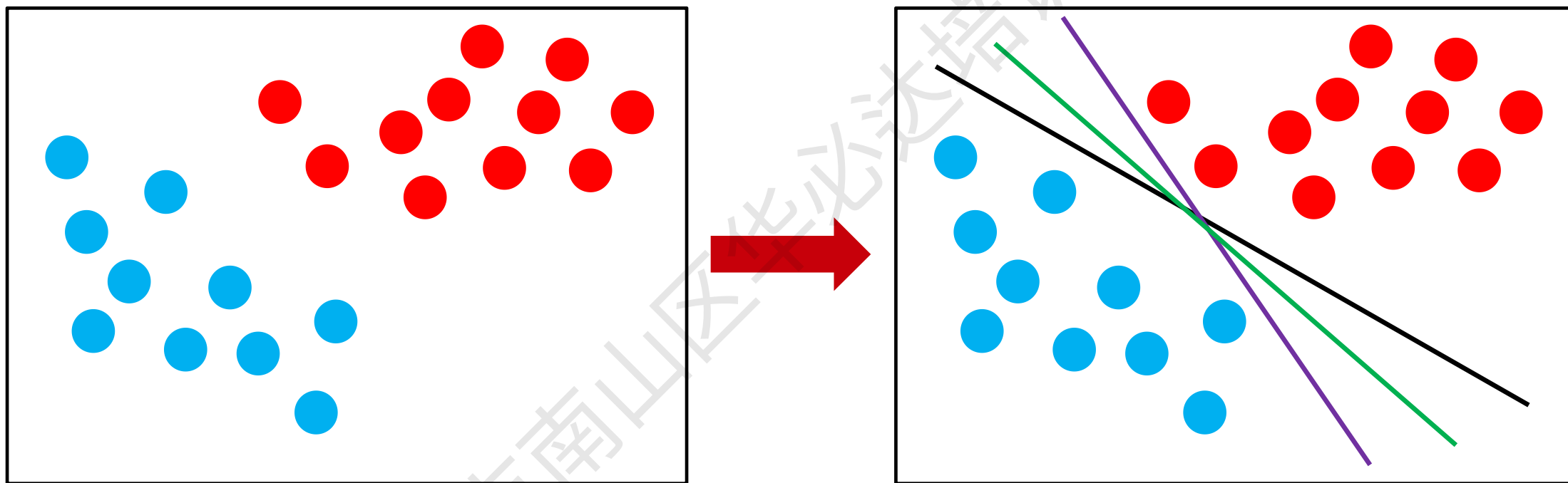
SVM支持向量机

你能用一根棍子将下面不同颜色的球分开吗？



SVM支持向量机

哪根棍子才最好呢？也就是说哪根棍子能够达到最好的泛化能力呢？

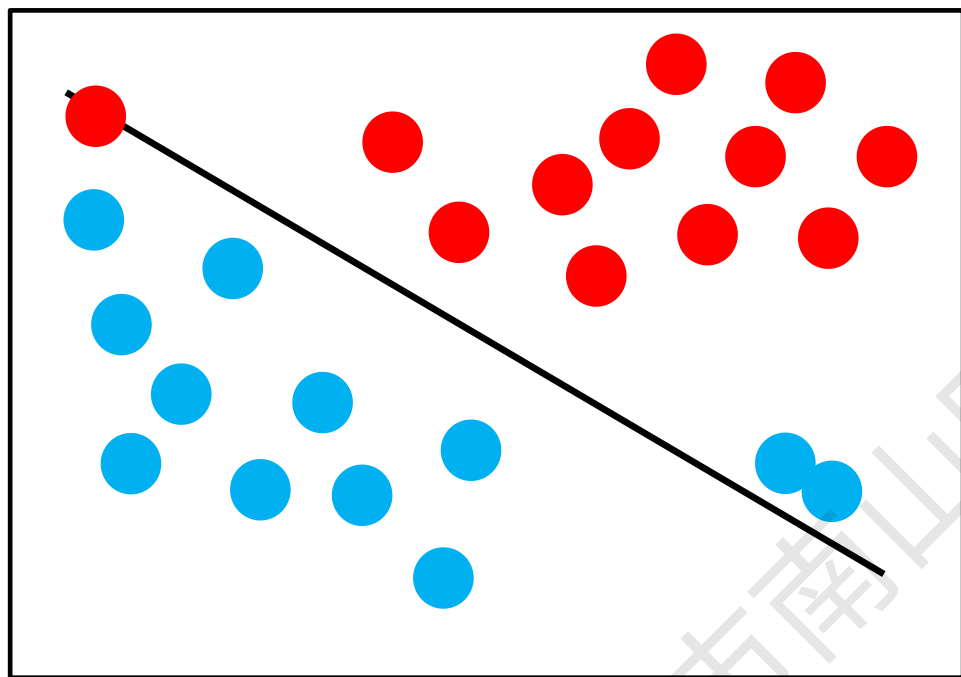


泛化能力：就是能使两类之间的空间大小最大

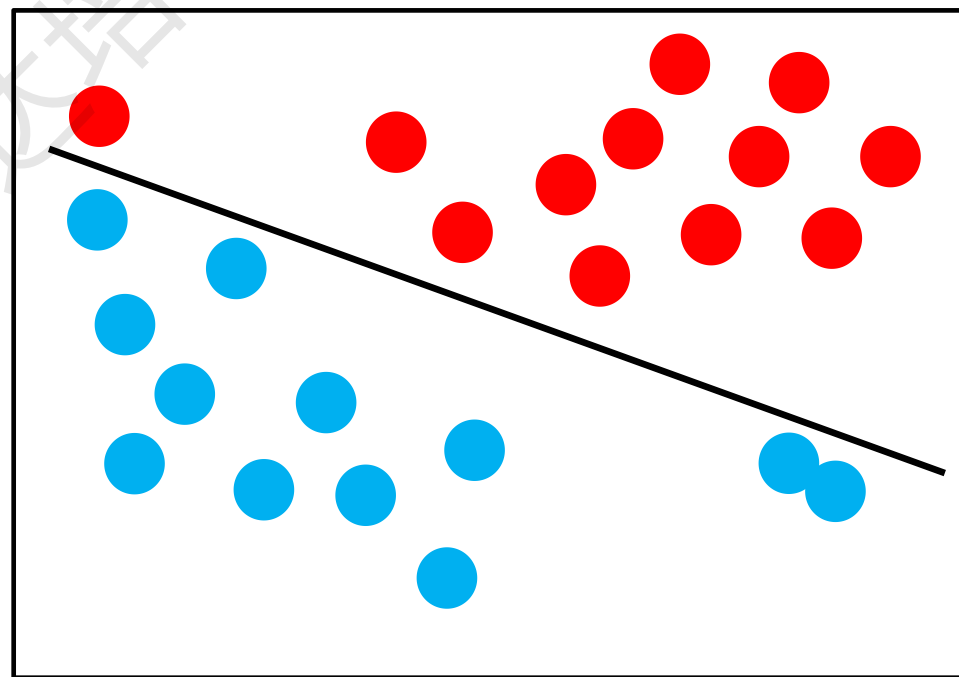
SVM支持向量机

要求：尽量在放更多球之后，仍然适用，那要怎么办？怎么放置棍子的位置呢？

放更多的球



调整棍的位置



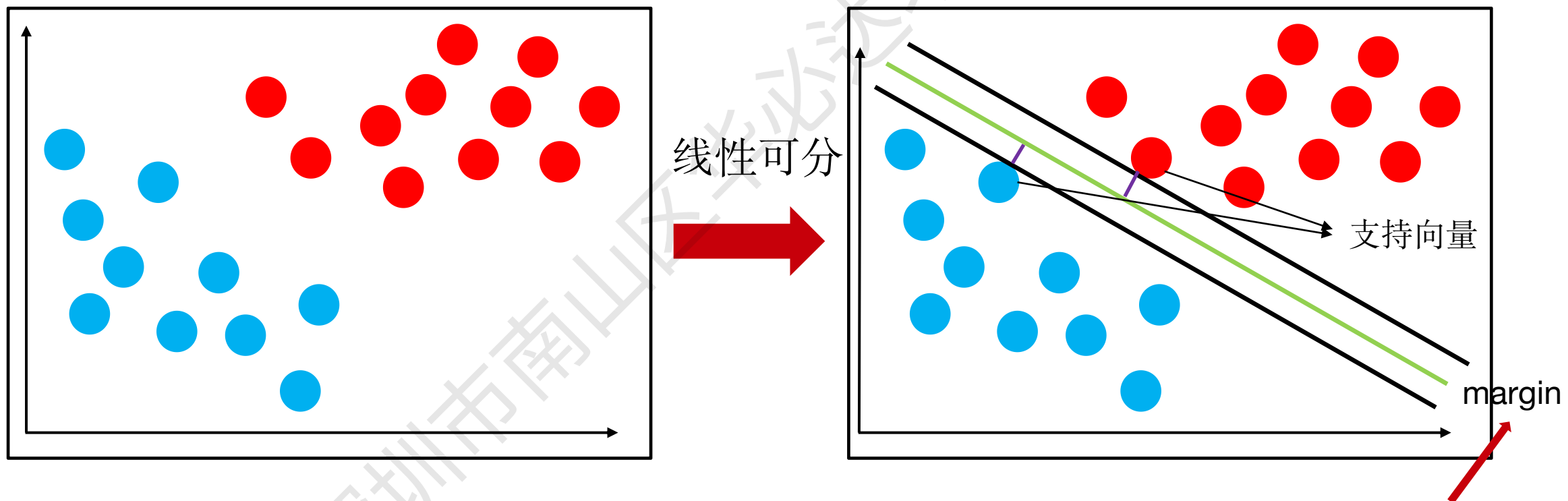
试图把棍放在最佳位置，好让在棍子的两边有尽可能大的间隙。这个间隙就是球到棍的距离。

支持向量机SVM本质模型是特征空间中最大化间隔的线性分类器，是一种二分类模型。

SVM支持向量机

SVM的最优化问题

直观看上去，应该去找位于两类训练样本的“**正中间**”画出分界线

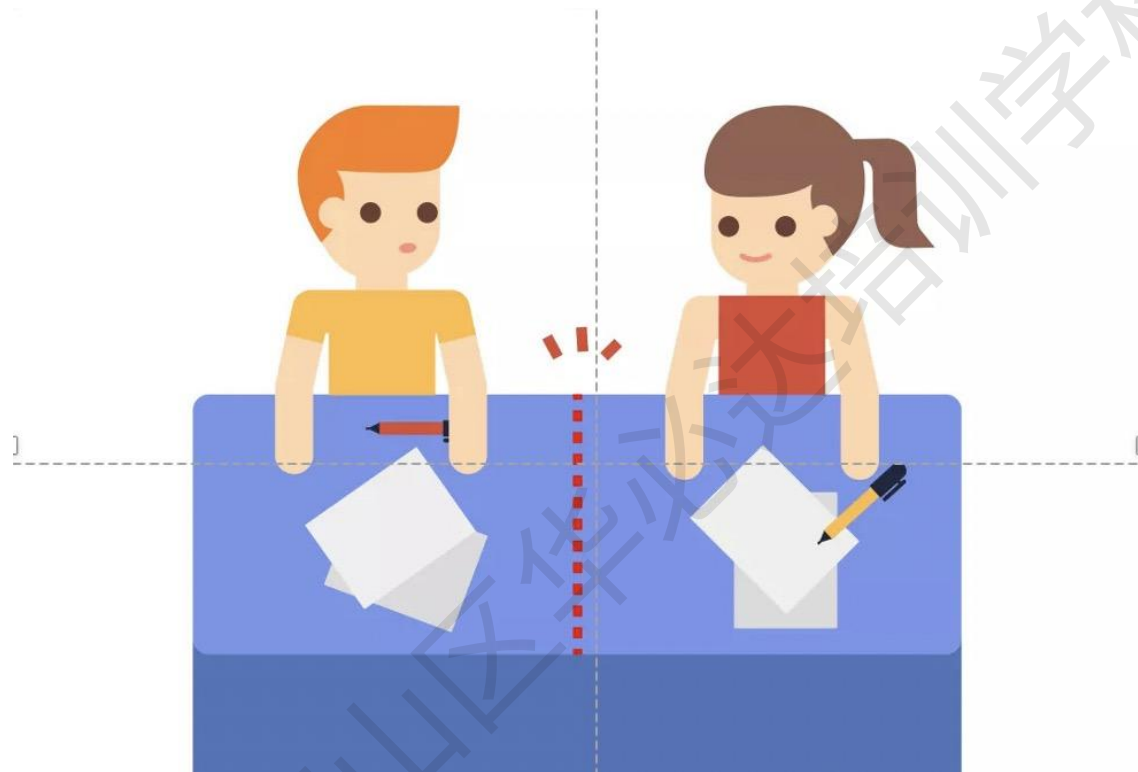


SVM要最大化margin

SVM尝试寻找一个最优的决策边界，距离两个类别的最近的样本最远

SVM支持向量机

“三八线”



这样的“三八线”传递出了以下几点重要的信息：

- 是一条直线；
- 能将桌面分为两个部分，具有分类功能，是一种二值分类；
- 位于课桌正中间，不偏向任何一方。



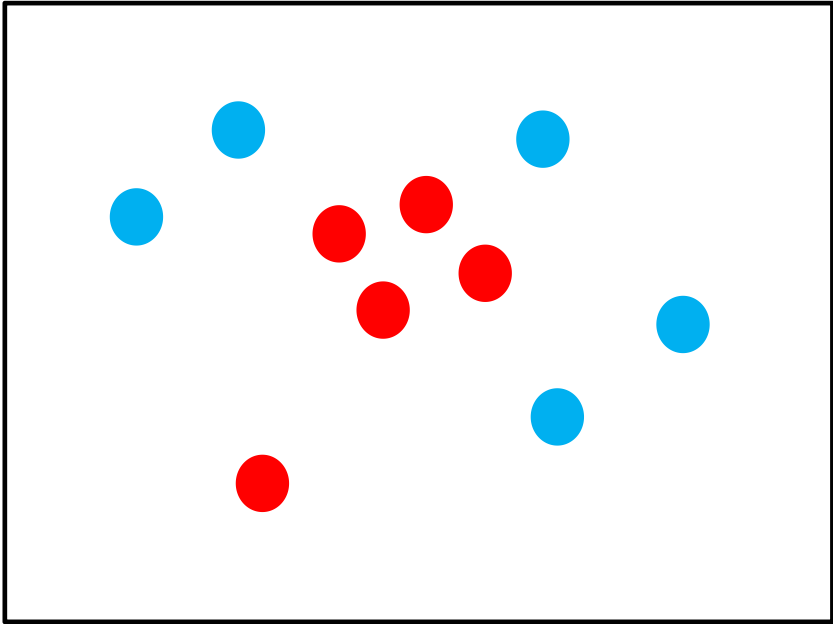
正是SVM分类器的中心思想

SVM支持向量机

支持向量机 (Support Vector Machine, SVM) 是一类按监督学习方式对数据进行二元的广义线性分类器，其决策边界是对学习样本求解的最大边距超平面

线性支持向量机学习算法如下：

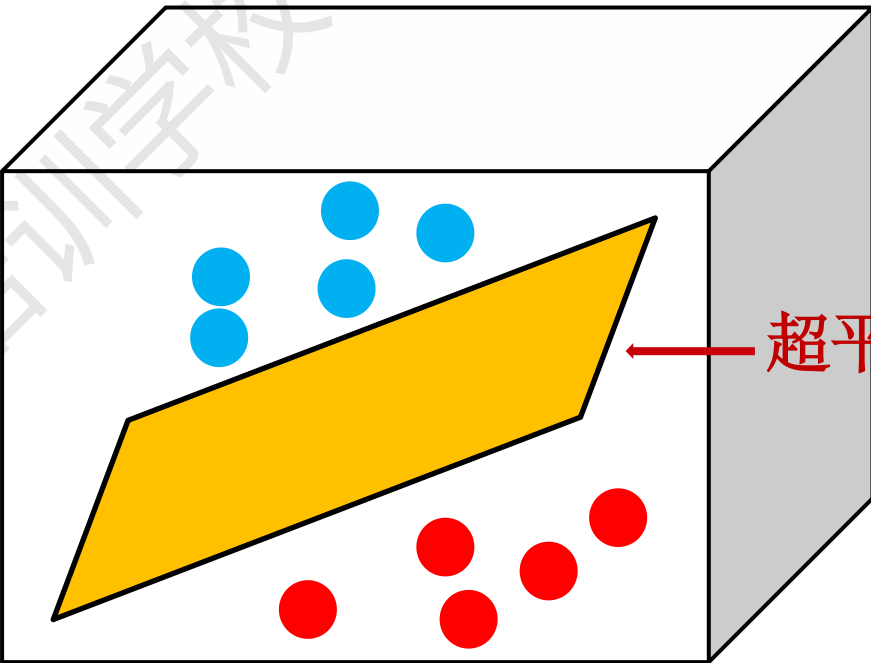
SVM支持向量机



将球都打入空中，
然后，拿一张纸，
插入两个球的中间

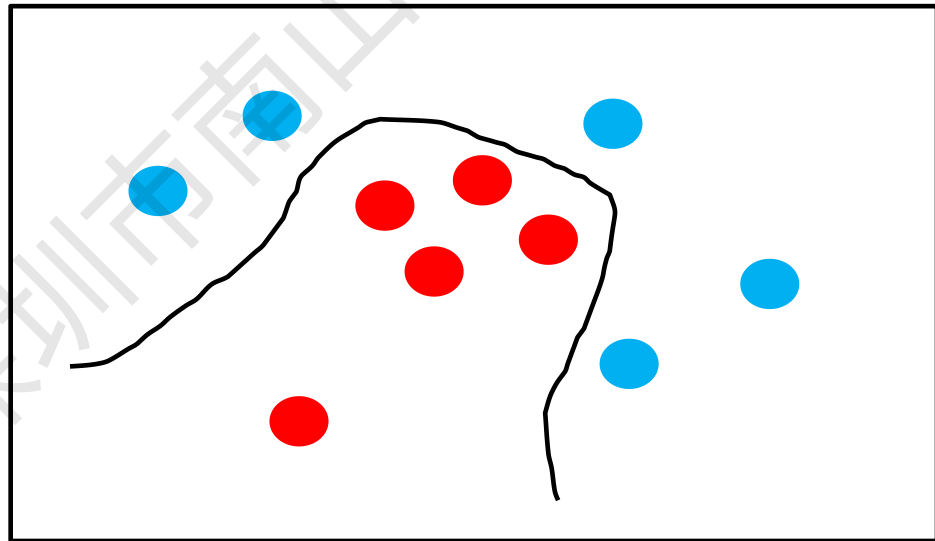


核化



超平面

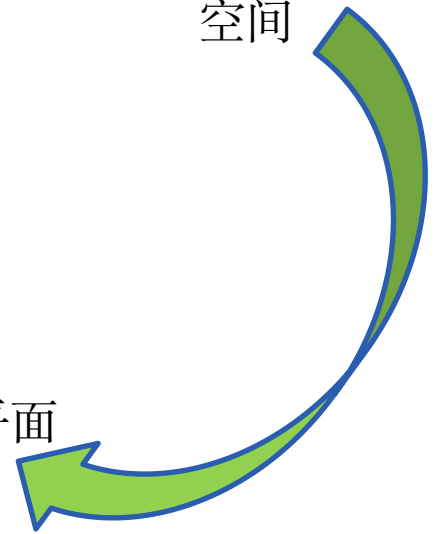
用一条曲线给分开了



空间

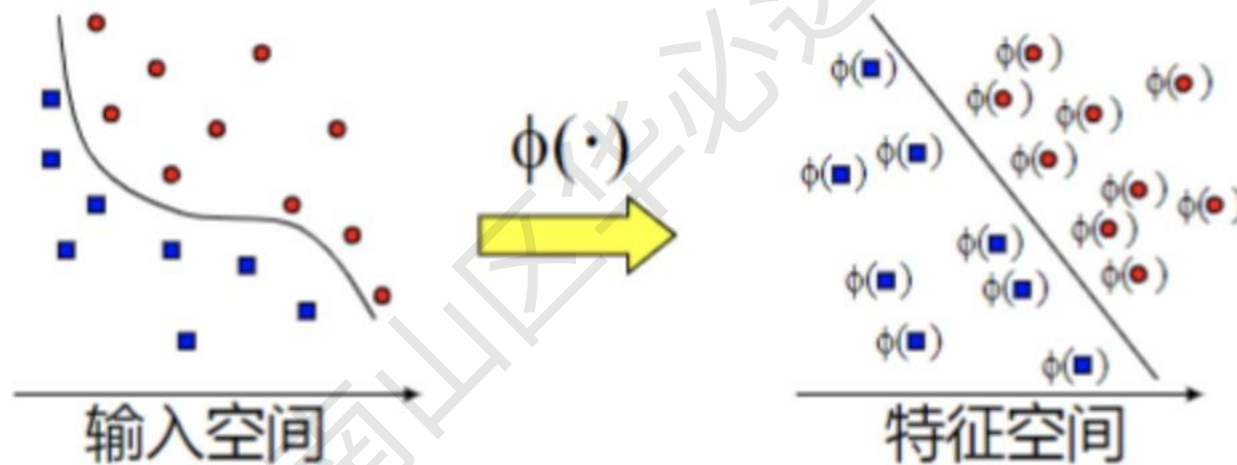
投影/转化

平面



SVM支持向量机

二维平面不可分，为了使数据可分，需要通过一个函数将原始数据映射到高维空间，从而使数据在高维空间很容易可分，**需要通过一个函数将原始数据映射到高维空间，从而使数据在高维空间很容易区分**，实现这一目标的函数称为**核函数**



对于任意空间输入的 x ， z 有：

$$K(x, z) = \phi(x) \cdot \phi(z)$$

SVM支持向量机

常用的核函数

线性核函数:

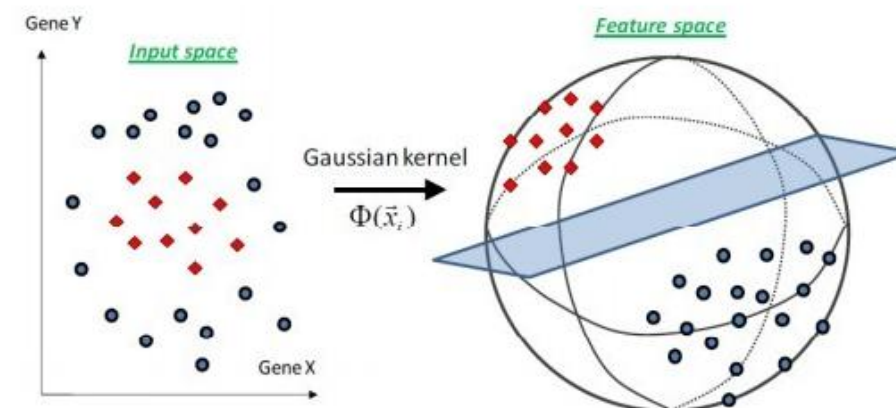
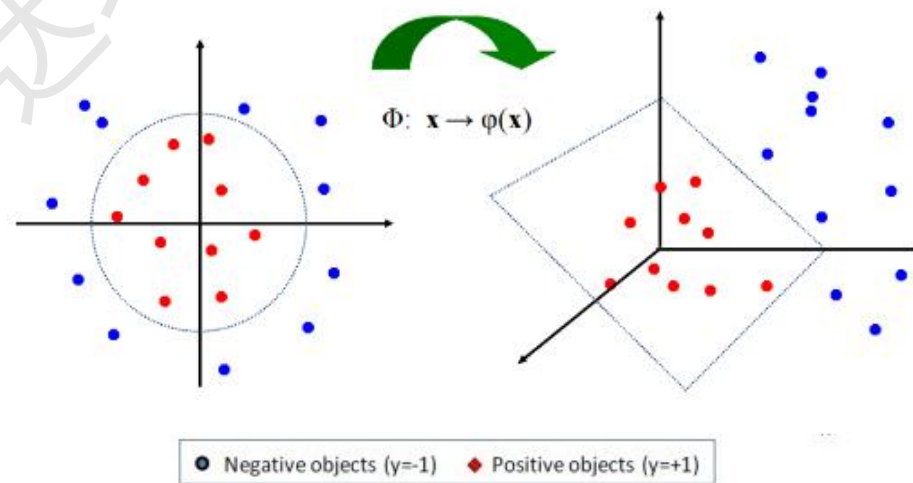
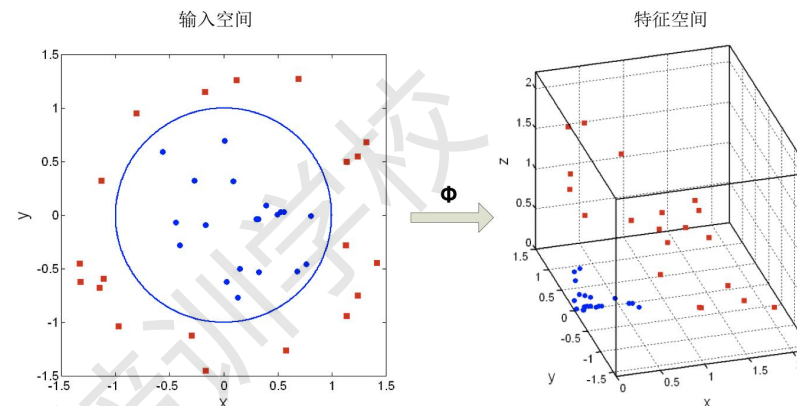
$$K(x, z) = |x, z| \quad (\text{内积})$$

多项式核函数:

$$K(x, z) = (x \cdot z + 1)^p$$

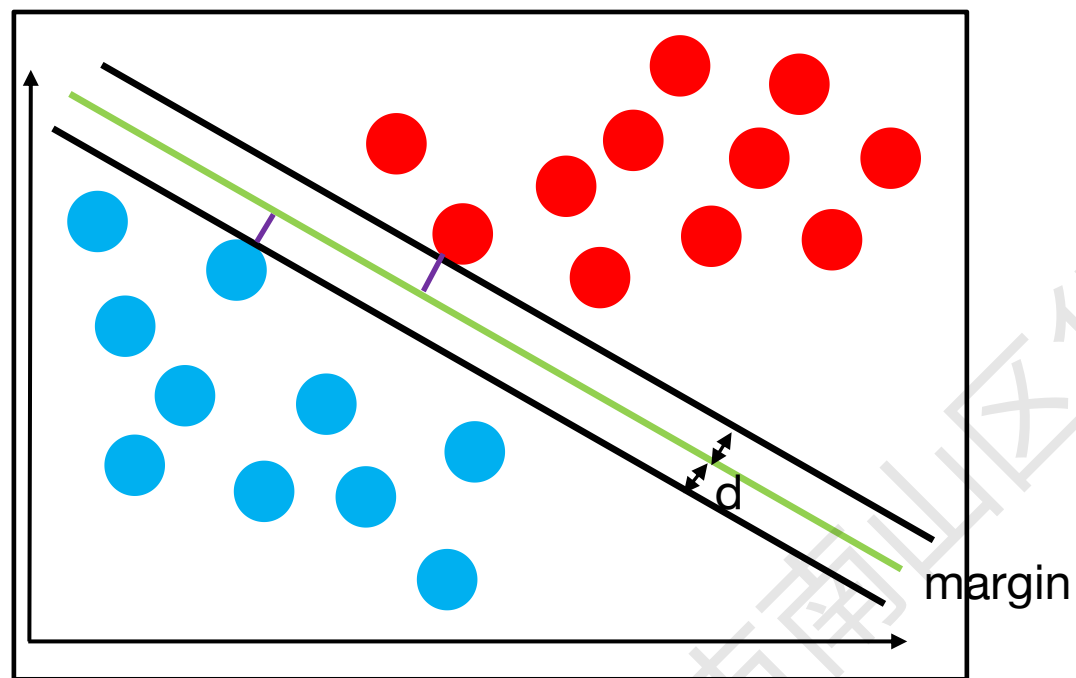
高斯核函数:

$$K(x, z) = e^{-\frac{|x-z|^2}{2\sigma^2}}$$



SVM支持向量机

数学描述



平面

SVM要最大化margin

$$\text{Margin} = 2d$$

SVM要最大化 d

SVM支持向量机

数学描述

点到直线的距离:

给出点 (x_0, y_0) ,

直线为 $Ax + By + C = 0$,

则点到直线的距离 r 为:

$$r = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$



点到平面的距离:

给出点 (x_0, y_0, z_0) ,

平面为 $Ax + By + Cz + D = 0$,

则点到平面的距离 r 为:

$$r = \frac{|Ax_0 + By_0 + Cz_0 + D|}{\sqrt{A^2 + B^2 + C^2}}$$



样本空间:

n 维空间点 $\mathbf{x}$, 超平面可通过线性方程描述为:

$$\mathbf{w}^T \mathbf{x} + b = 0,$$

其中, $\mathbf{w} = (w_1; w_2; \dots; w_d)$ 为超平面的法向量, 决定超平面的方向, b 为位移项, 决定超平面与 n 维空间中的原点直接的距离

n 维空间中的任意一点 $\mathbf{x}$ 到超平面 $(\mathbf{w}, b)$ 的距离为:

优化 $\rightarrow$

$$r = \frac{|\mathbf{w}^T \mathbf{x} + b|}{|\mathbf{w}|}$$

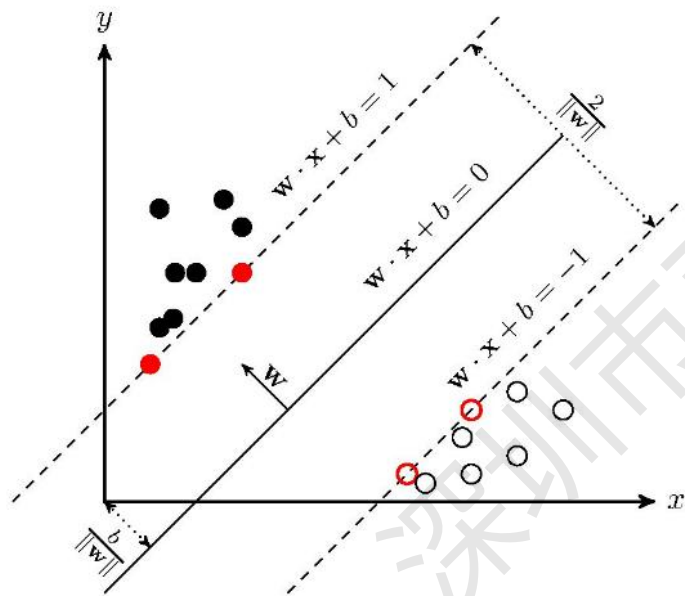
$$|\mathbf{w}| = \sqrt{w_1^2 + w_2^2 + \dots + w_d^2}$$

SVM支持向量机

数学描述

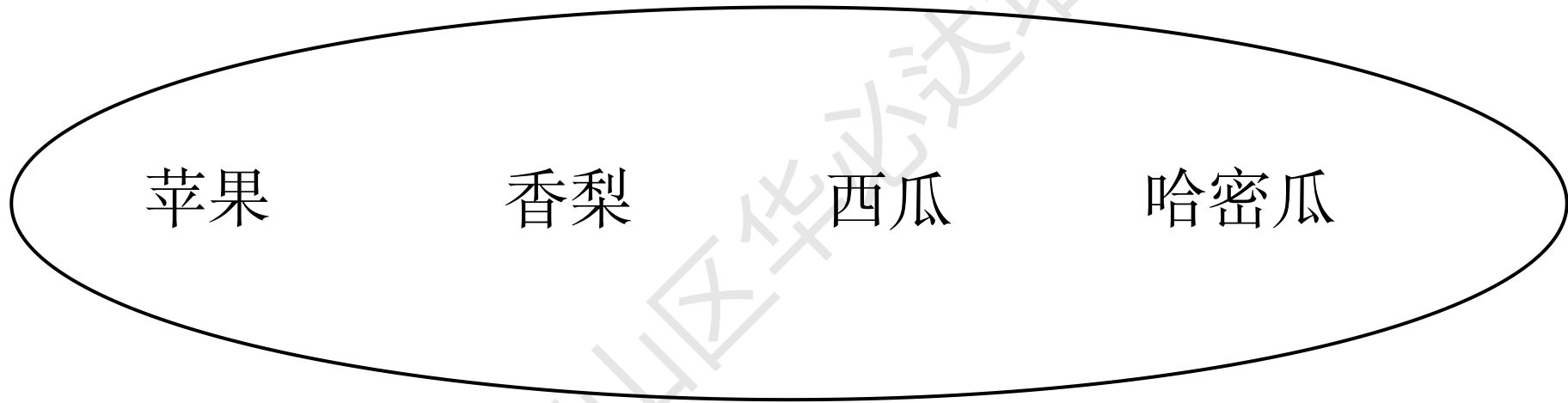
最大间隔的最优化问题最终转化为:

$$\min_{w,b} \frac{1}{2} |w|^2$$
$$\text{s. t. } y_i(w^T x + b) \geq 1, \quad i = 1, 2, \dots, m$$



这就是支持向量机 (Support Vector Machine, SVM) 背后的最优化问题

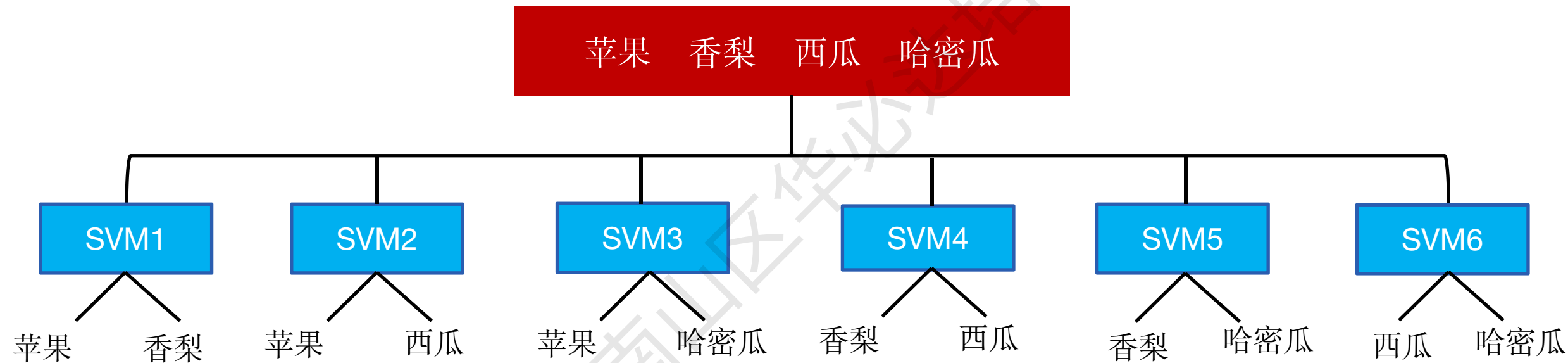
SVM支持向量机



怎么用SVM进行分类?

SVM支持向量机

萝卜白菜各有所爱



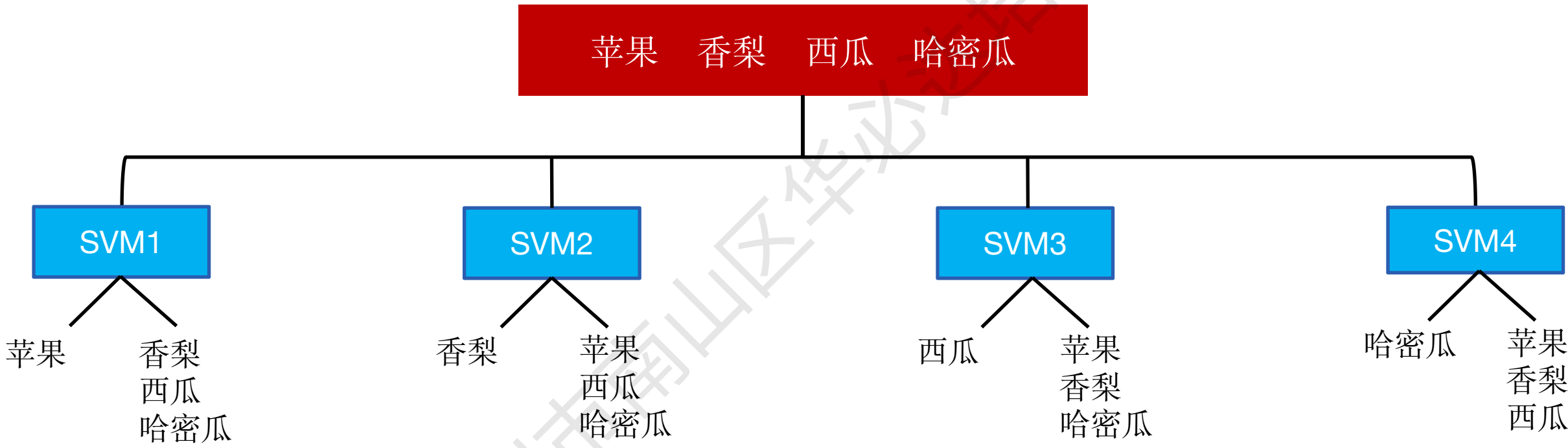
在每两类水果样本之间设计一个SVM模型，通过设计6个SVM分类器最终将4种水果区分出来

假设样本类别有 m 个，则总共需要设计 $m(m-1)/2$ 个SVM分类器

当水果类别达1000种时，则总共需要设计 $1000 \times (1000-1)/2 = 499500$ 个SVM分类器，**算法计算复杂度相当高。**

SVM支持向量机

萝卜白菜各有所爱



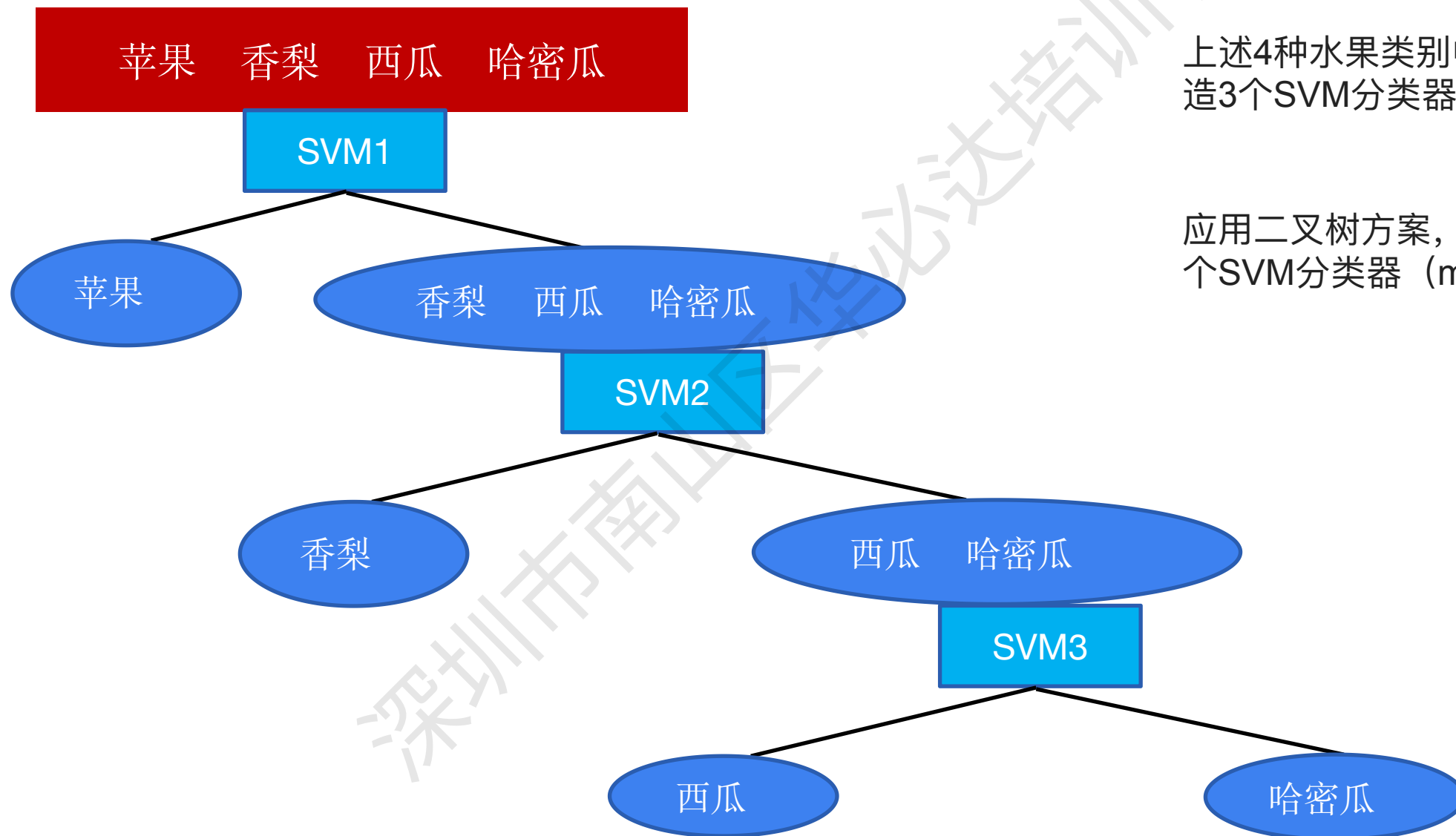
选定其中一种类别样本单独作为一个类别，除该类别外的其余类别样本则归为另一个类别，这样就达到了SVM二分类的要求。

有多少种类别的样本，就需构造多少个SVM分类器。

SVM支持向量机

SVM-多分类

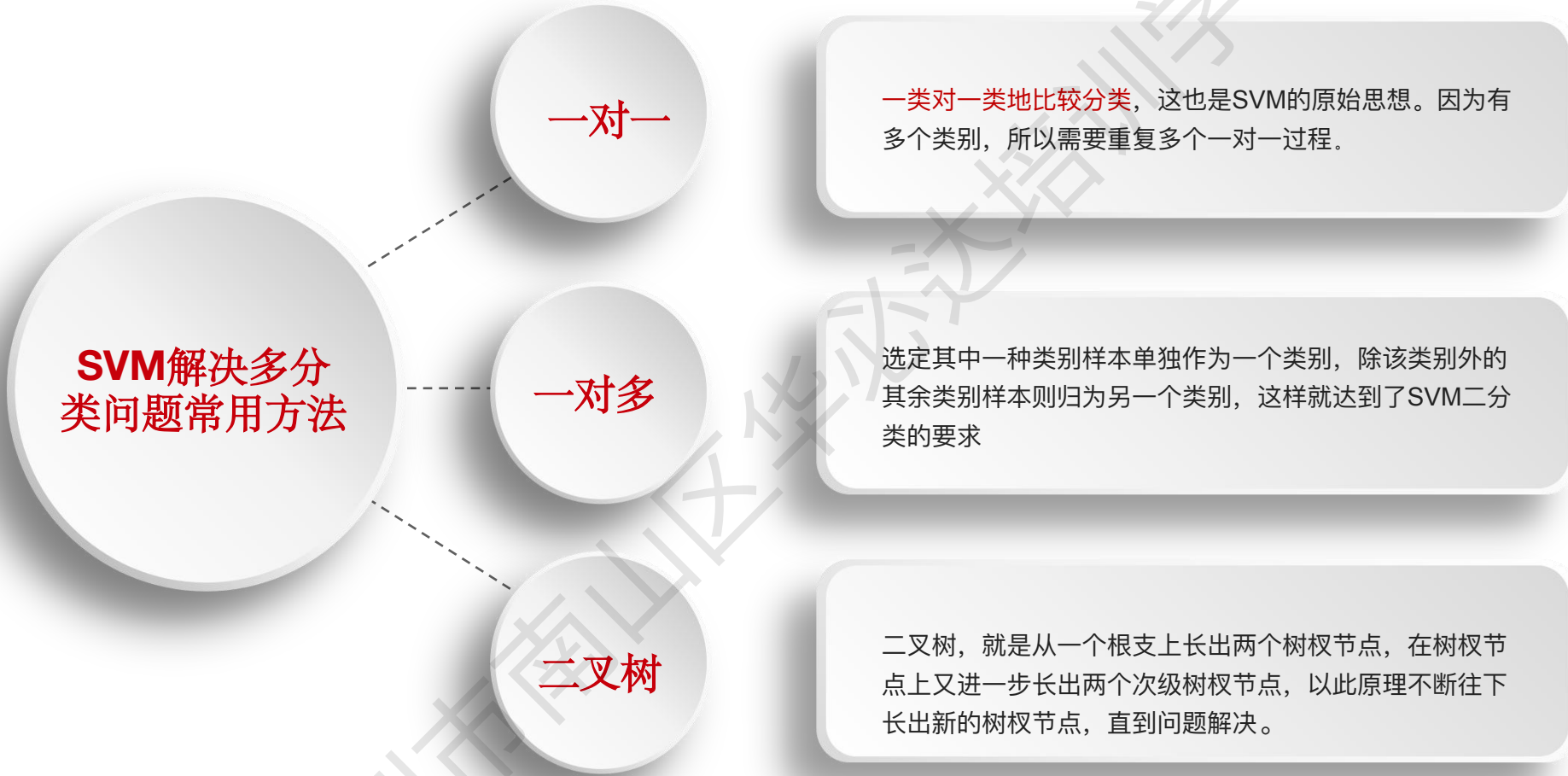
萝卜白菜各有所爱



上述4种水果类别中，总共需要构造3个SVM分类器来完成分类

应用二叉树方案，总共需要构造 $m-1$ 个SVM分类器（ m 为样本类别数量）

SVM支持向量机

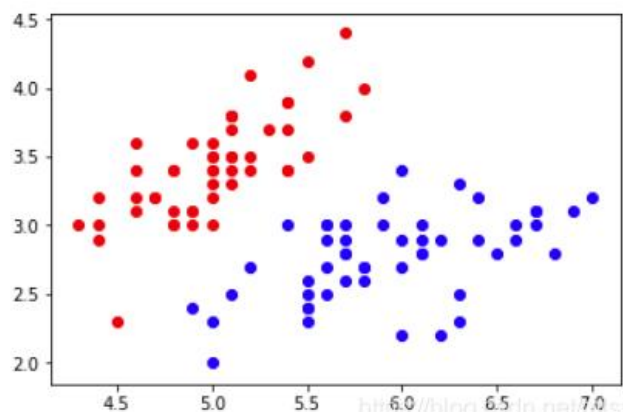


在应用中，随着样本类别的增多，训练的难度和复杂度也随之增加。**如何减轻计算量和降低计算难度**，是SVM在多分类应用中需要着重考虑的难点之一。

SVM支持向量机

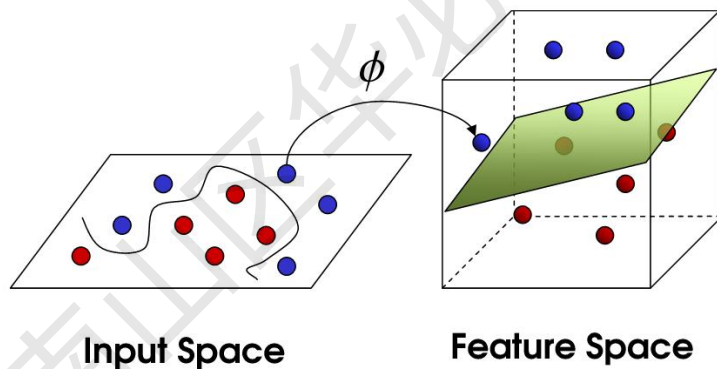
基于Python编程实现SVM分类算法

步骤1: 基于Python加载**鸢尾花**数据集，并查看散点图分布

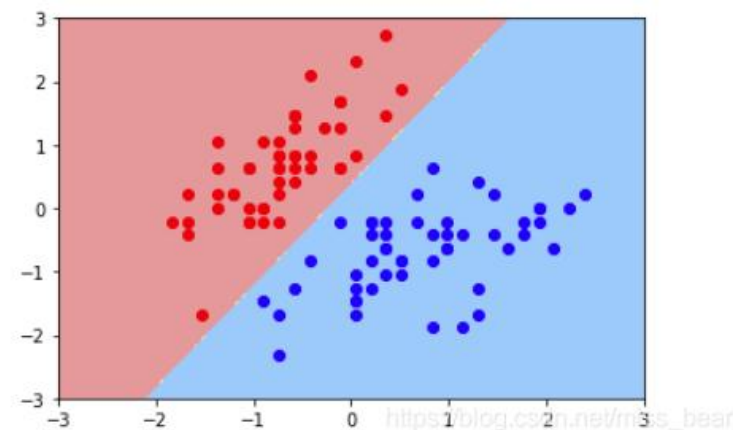


步骤2: 训练SVM分类器

```
svc = SVC()  
svc.fit(X,y)
```



步骤3: 绘制决策边界





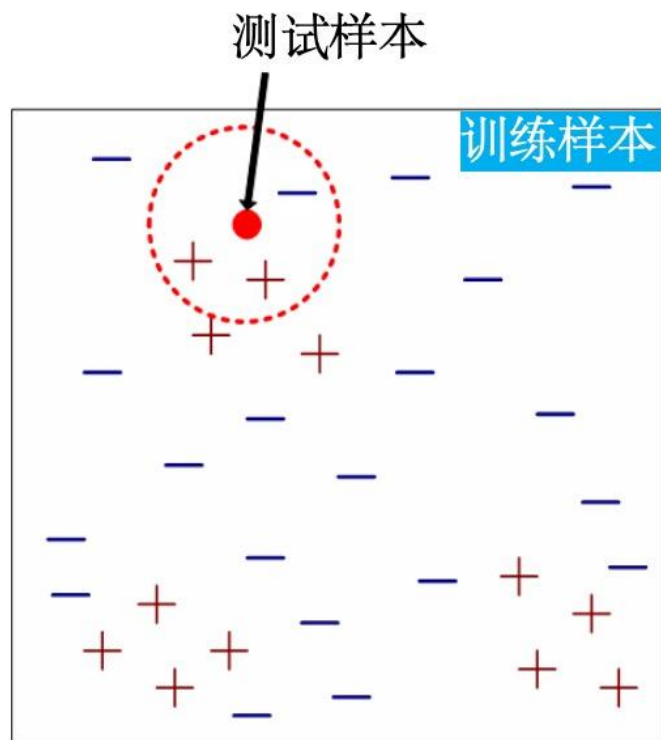
Scikit-learn(sklearn)

<https://scikit-learn.org/stable/modules/classes.html>

```
class sklearn.svm.SVC(C=1.0, kernel='rbf', degree=3, gamma='scale', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr', break_ties=False, random_state=None)
```

- ❑ *C*是超参数，控制模型复杂度，*c*越大，容忍度越小，*c*越小，容忍度越高，泛化能力较强。
- ❑ *kernel* 核函数，默认是rbf，可以是‘linear’，‘poly’，‘rbf’，‘sigmoid’，‘precomputed’
 - linear: 线性核函数
 - poly: 多项式核函数
 - rbf: 高斯核函数
 - sigmoid: Sigmoid核函数，类似使用一个一层的神经网络
 - precomputed : 提供已经计算好的核函数矩阵，不常用
- ❑ *degree*多项式poly函数的维度，默认是3，选择其他核函数时会被忽略。
- ❑ *decision_function_shape*, ‘ovo’, ‘ovr’ or None, default=None3

k-近邻 (KNN) 分类

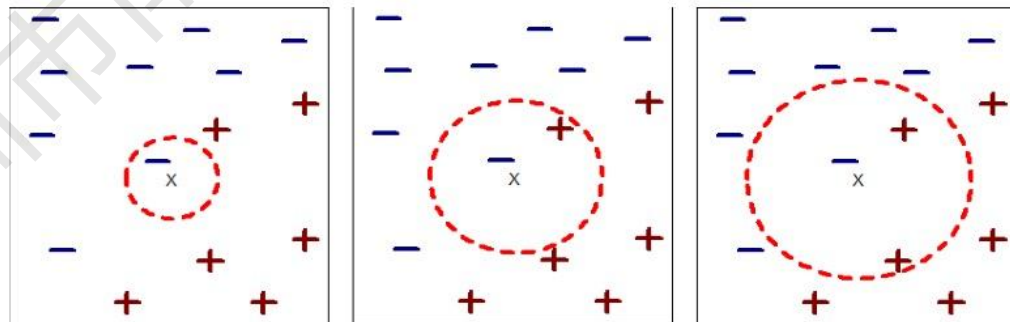


- 给定 ---
 - 训练样本('+'/'-'两类)
 - 相似性度量(欧氏距离)
 - k (考虑 k 个最近的邻居)

- 对测试样本分类 ---
 - 计算与所有测试样本的距离
 - 找出 k 个最近的邻居
 - 用**多数投票**方法确定类别

👉 没有“建模”过程，是懒分类器

k 值的影响与
多数投票机制



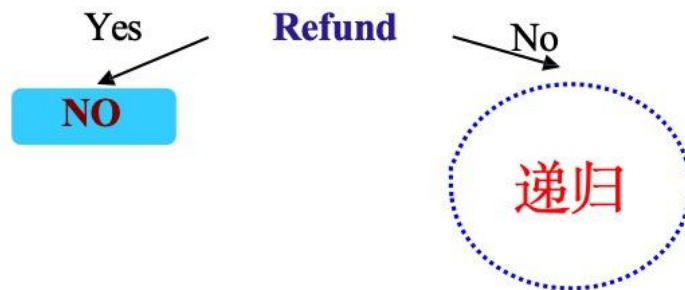
决策树分类

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

步骤1: 选择一个**分割属性**
根据它的取值创建分支



步骤2: 为每个分支建立一颗子树
(1) 若它是“纯的”，则创建叶子；
(2) 否则，递归建树（执行步骤1、2）



实质上，分割属性的作用就是对数据进行了“分割”...

聚类

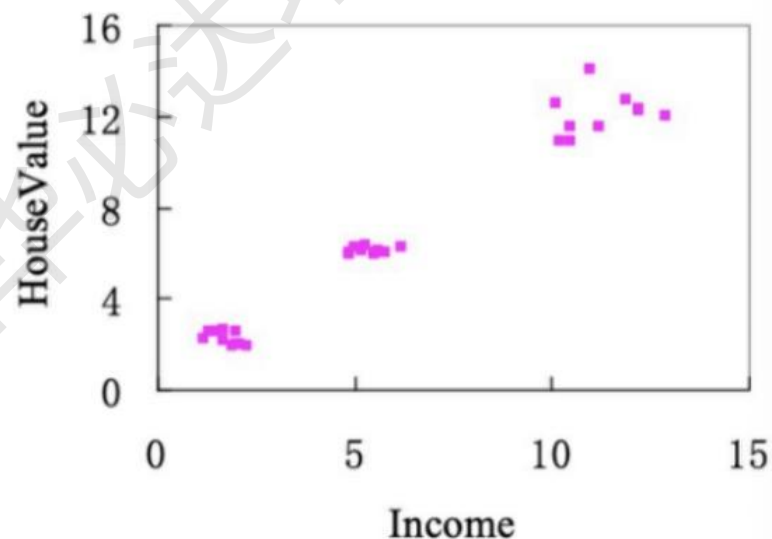
■ 聚类(clustering)

— 给定一个样本集合, 将样本划分为若干个子集, 使得

- ✦ 在同一个子集内的样本是彼此相似的
- ✦ 不同子集的样本之间是很不相似的

— 这样的子集称为**簇(cluster)**

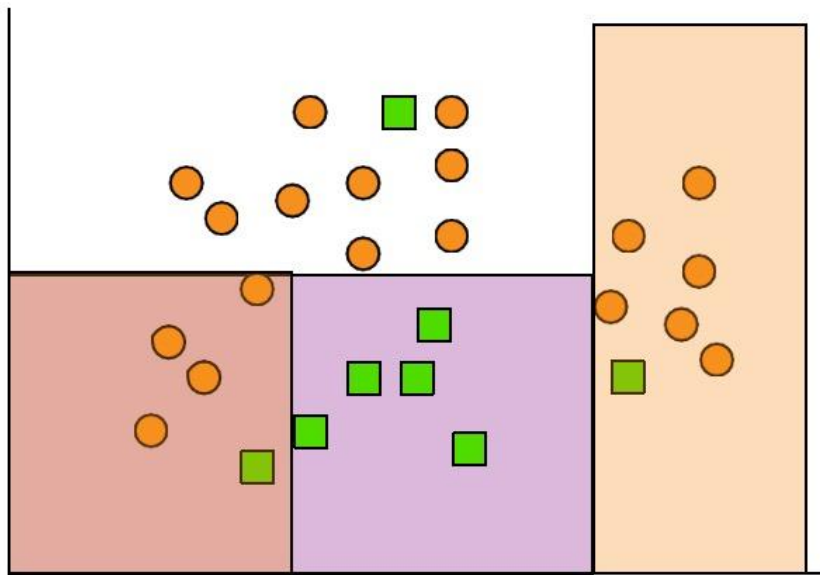
- 聚类是一种**无监督**的机器学习方法
- 是一种**描述性**的数据挖掘方法



将样本（像素）分为两个簇：

- ✓ 一个簇对应“前景色”
- ✓ 另一个对应“背景色”

分类 VS 聚类

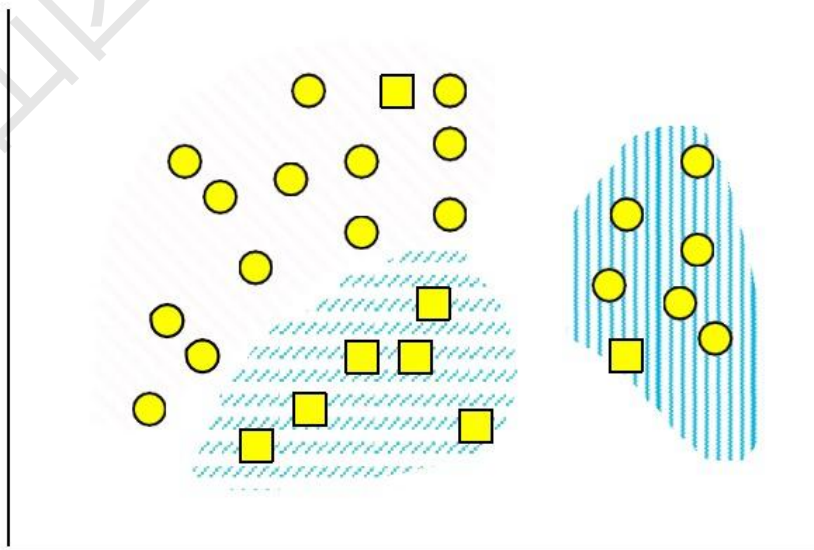


分类(Classification):

- ❖ 有监督的学习
- ❖ 从已经分类好的样本集学习到一种模型用于预测样本的类别

聚类(Clustering):

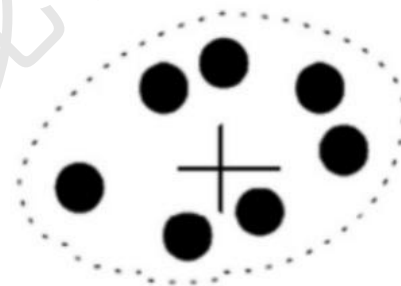
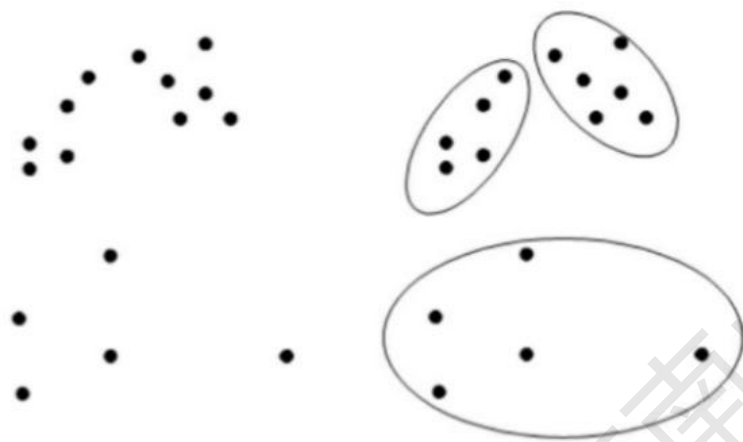
- ❖ 无监督的学习
- ❖ 找到样本的自然分组



K-Means聚类

- 基本原理

给定 N 个样本和聚类数 K ，将样本划分为 K 个分组，每个分组中的样本紧密地分布在其“中心”附近。

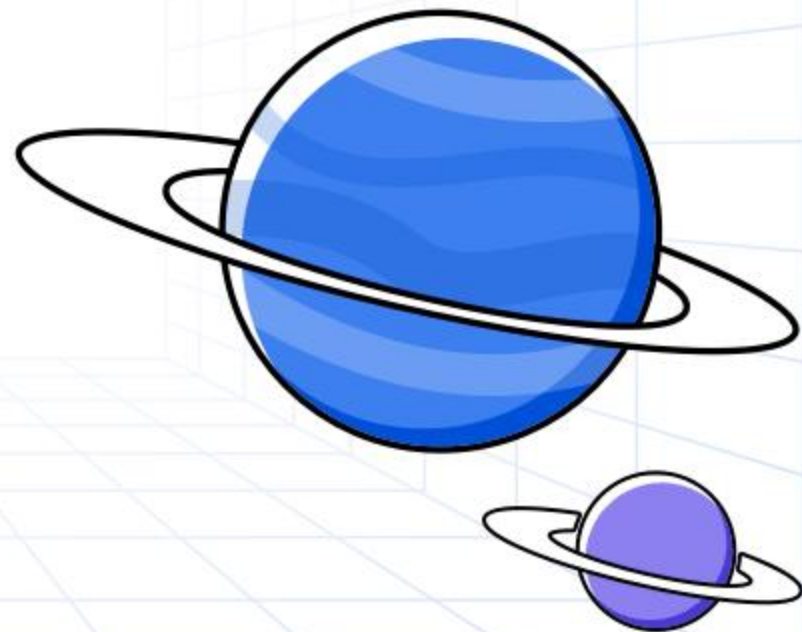


每个分组有一个“中心”
(平均值; means)

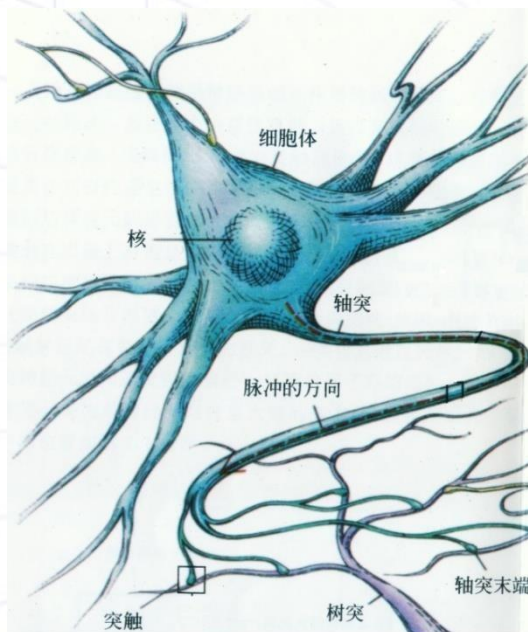
目标优化函数: $\min J(C) = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|_2^2$

04

深度学习

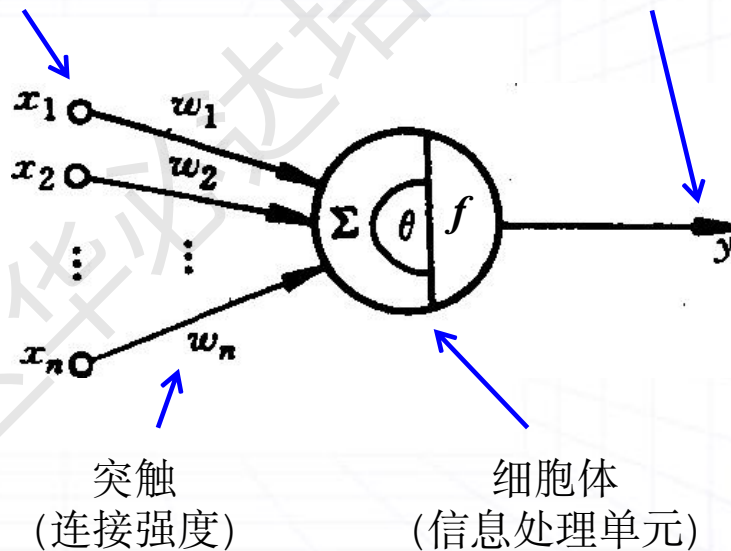


人工神经元

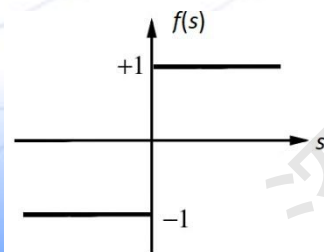


n 个树突
(n 个输入)

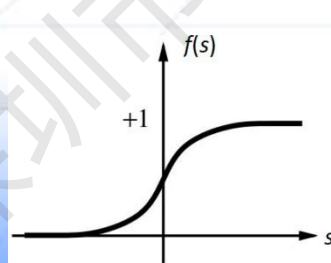
1个轴突
(1个输出)



符号激励函数



Sigmoid函数

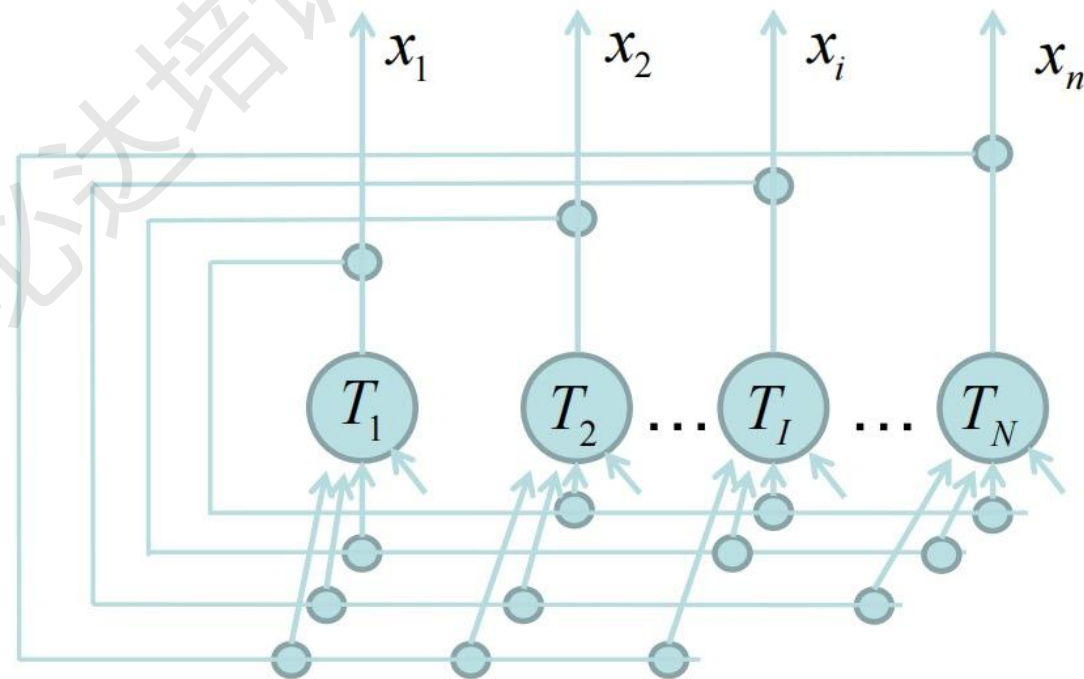


$$y = f\left(\sum_{i=1}^n w_i \cdot x_i - \theta\right)$$

感知器 (M-P模型)

人工神经元

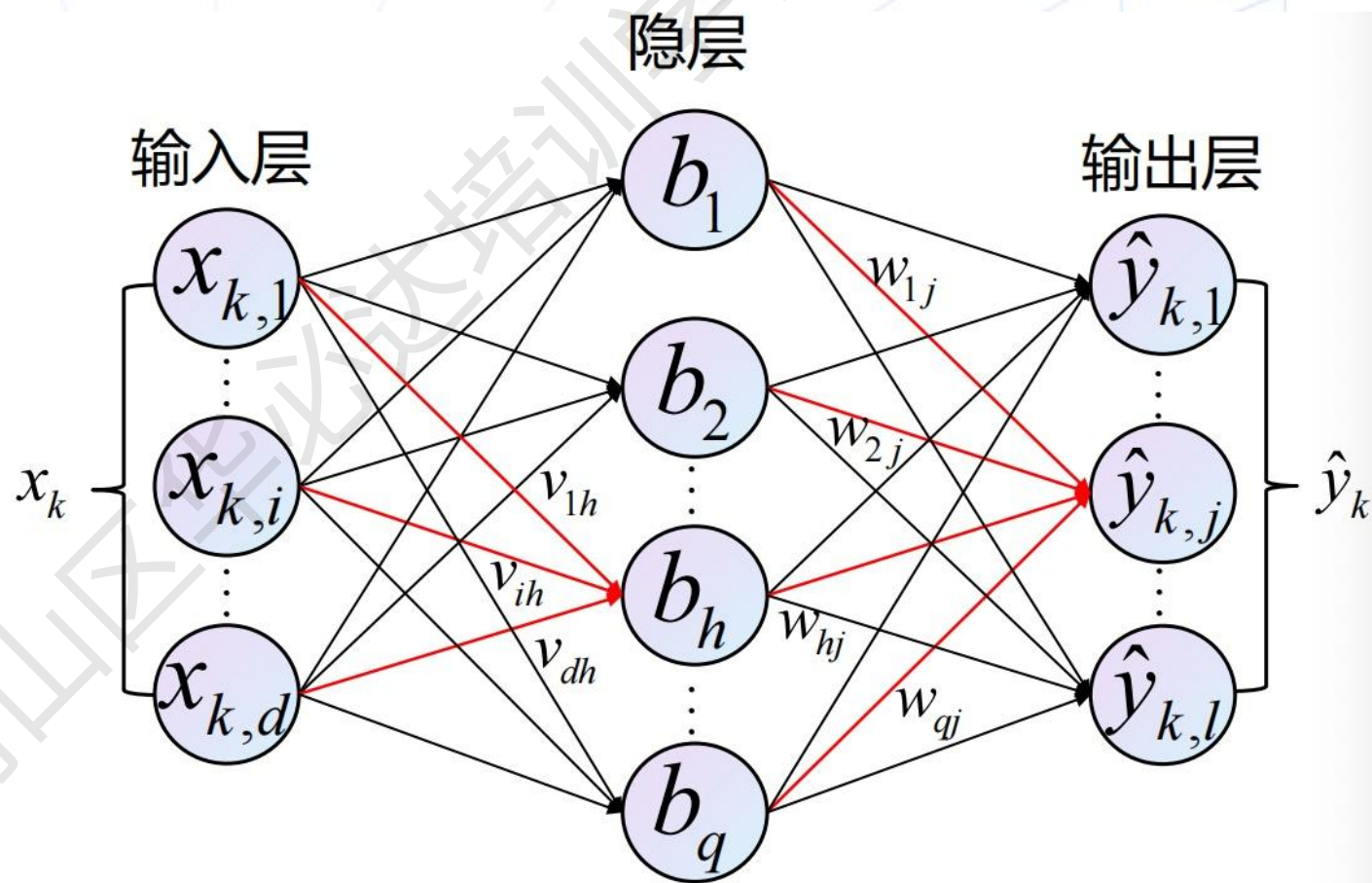
1982年，加州理工学院J.J.Hopfield教授提出了Hopfield神经网络模型，引入了计算能量概念，给出了网络稳定性判断。



离散Hopfield神经网络模型

人工神经元

1986年, Rumelhart和McClelland为首的科学家提出了BP (Back Propagation) 神经网络的概念, 是一种按照误差逆向传播算法训练的多层前馈神经网络, 目前是应用最广泛的神经网络。



BP神经网络模型

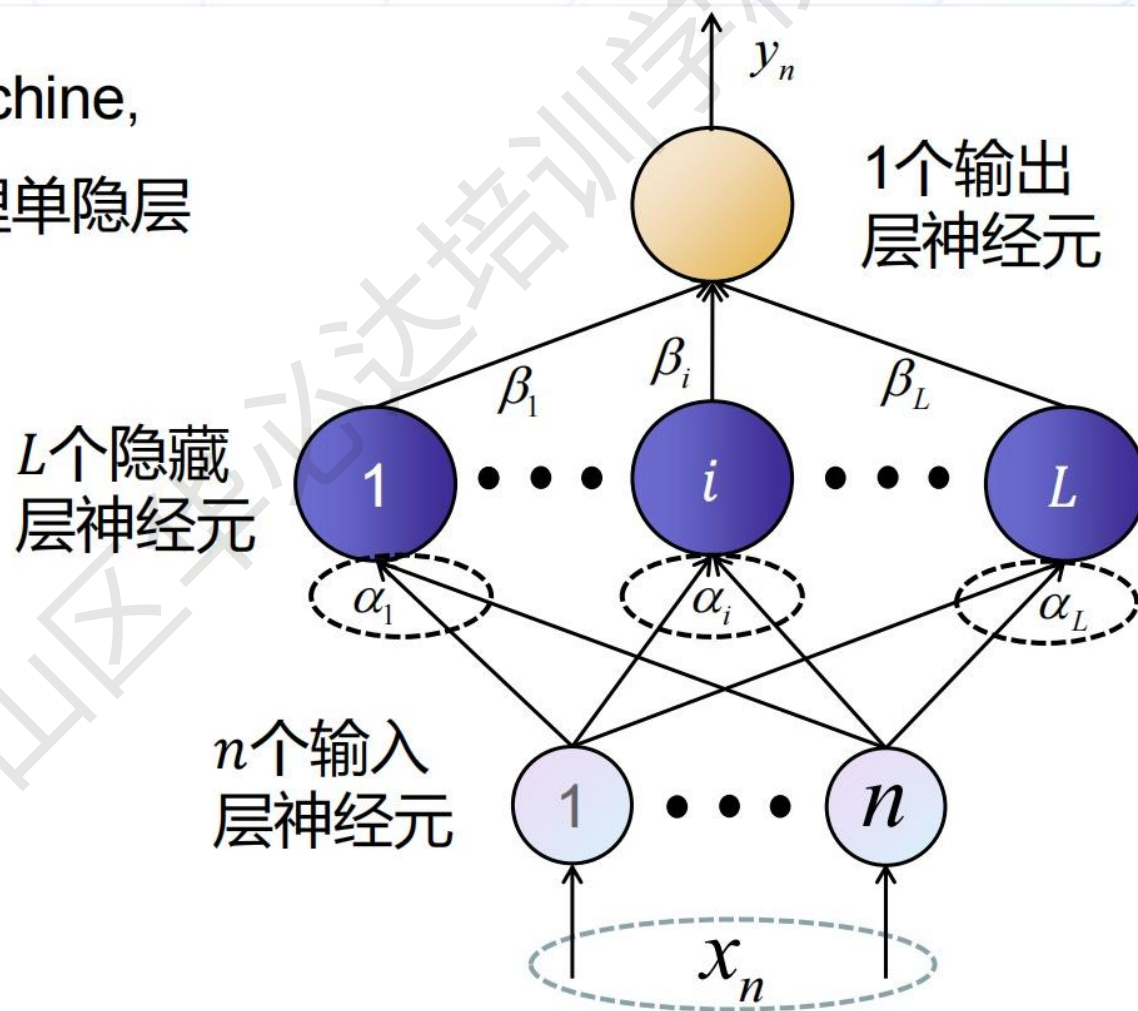
人工神经元

极限学习机(Extreme Learning Machine, ELM), 是由黄广斌提出的用于处理单隐层神经网络的算法

随机初始化输入权重 α_i 和偏置, 只求解输出权重值 β_i 。

优点:

- 1.学习精度有保证
- 2.学习速度快



感知机算法

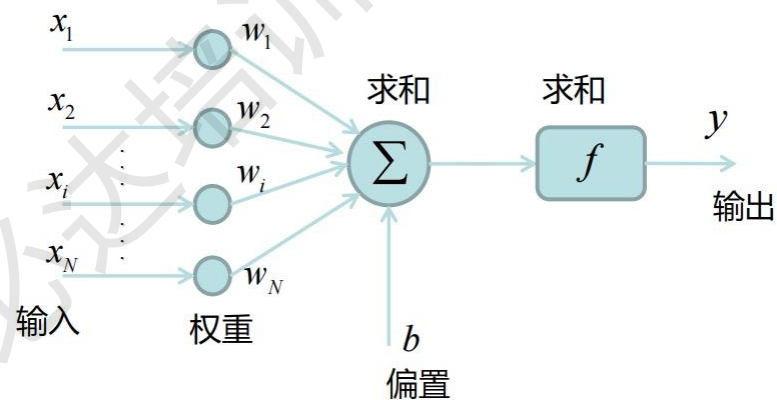
感知机 (Perceptron) 是二分类问题的线性分类模型。

用 $X \in R^{n \times d}$ 表示数据集, 用 Y 表示标签。

需要学习的目标函数是

$$f(x) = \text{sign}(w^T x + b)$$

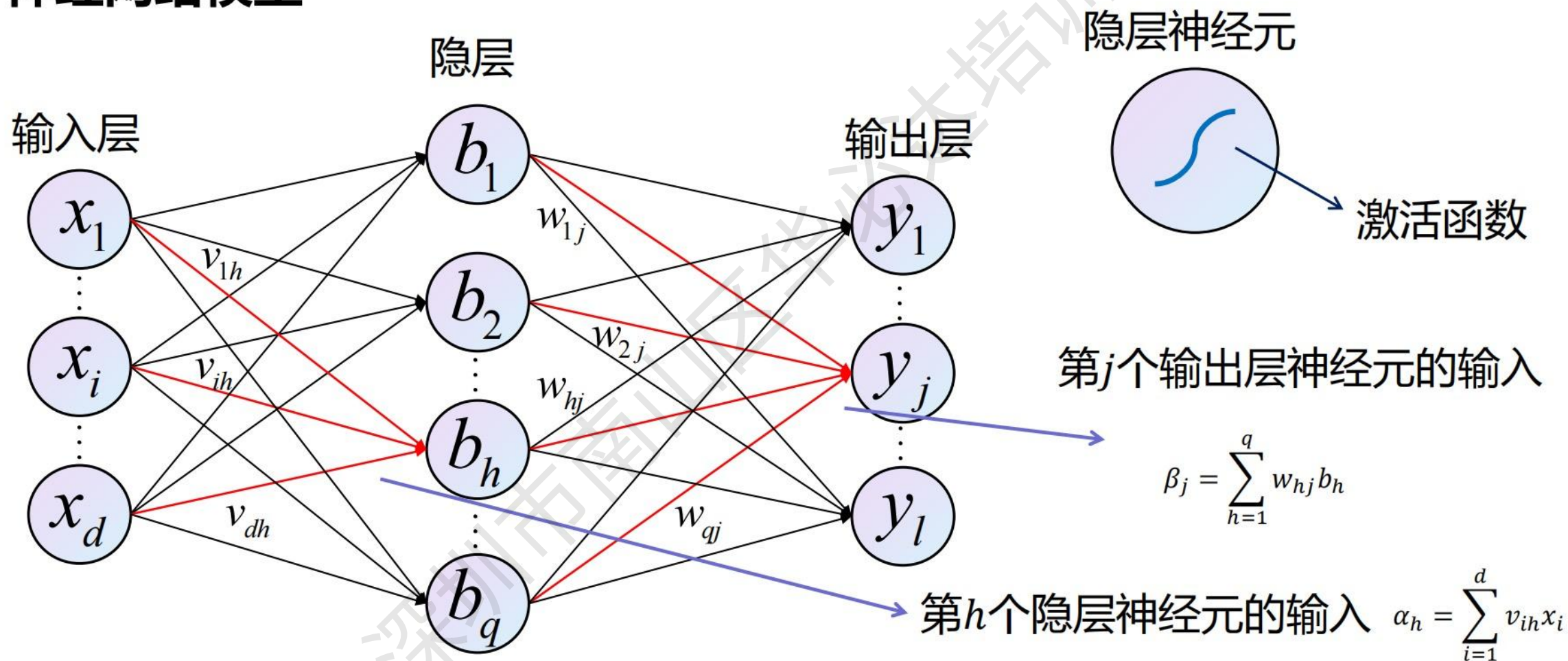
从一堆输入输出中学习模型参数 w 和 b 。



单层感知机只能处理
线性问题, 无法处理
非线性问题! !

BP算法

神经网络模型



BP算法

激活函数

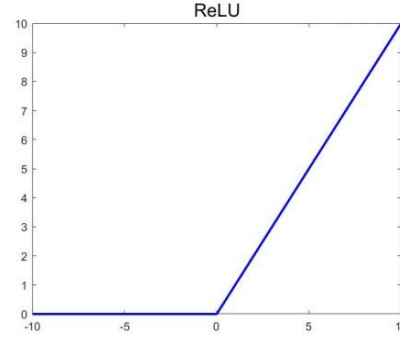
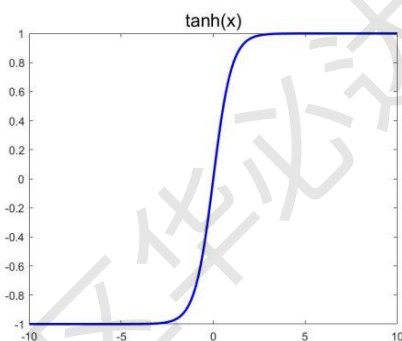
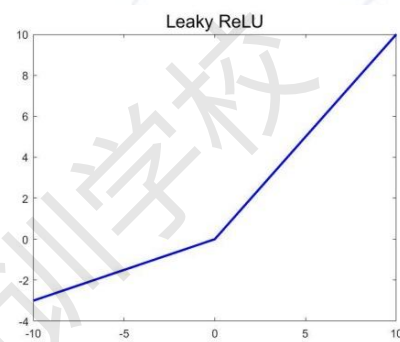
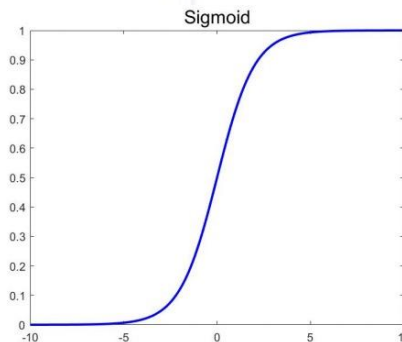
常见激活函数选择:

Sigmoid 函数

Tanh 函数

ReLU 函数

Leaky ReLU函数



最常用**Sigmoid**函数的优缺点:

优点:

- 1.函数处处连续, 便于求导
- 2.可将函数值的范围压缩至 $[0,1]$, 可用于压缩数据, 且幅度不变
- 3.便于前向传输

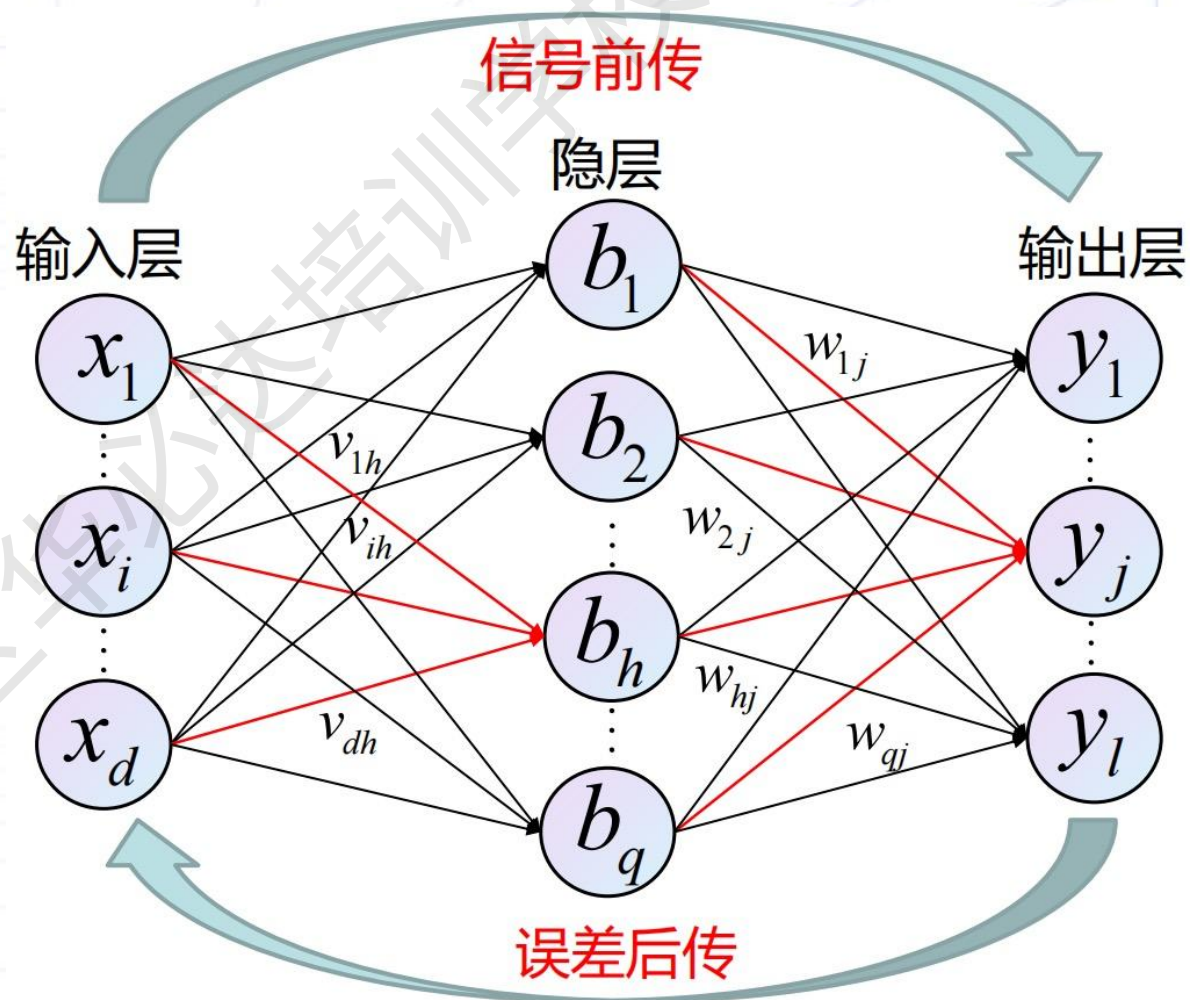
缺点:

- 1.在趋向无穷的地方, 函数值变化很小, 容易出现梯度消失, 不利于深层神经的反馈传输
- 2.幂函数的梯度计算复杂
- 3.收敛速度比较慢

BP算法

算法流程:

1. 将输入样本提供给输入层神经元
2. 逐层将**信号前传**至隐层、输出层, 产生输出层的结果
3. 计算输出层误差
4. 将**误差反向传播**至隐藏层神经元
5. 根据隐层神经元对连接权重和阈值进行调整
6. 上述过程循环进行, 直至达到某些停止条件为止



卷积神经网络 (CNN)

例子1：求扔两个色子结果之和为4的概率



卷积神经网络 (CNN)

例子1: 求扔两个色子结果之和为4的概率

f

1	2	3	4	5	6
---	---	---	---	---	---

f 表示第一枚骰子

$f(1)$ 表示投出1的概率

$f(2)$ 、 $f(3)$ 、 $\dots$ 以此类推

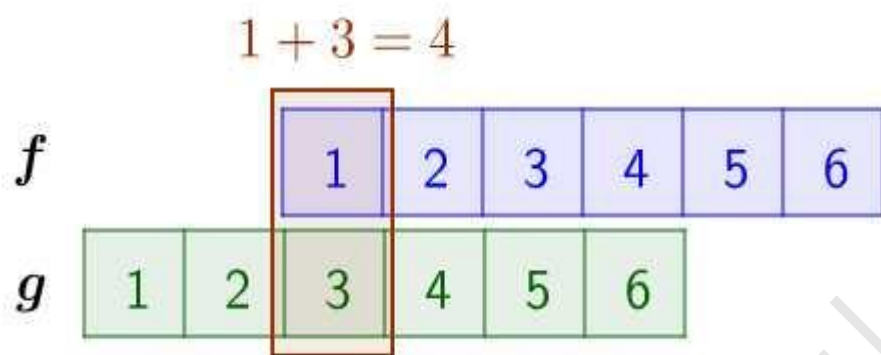
g

1	2	3	4	5	6
---	---	---	---	---	---

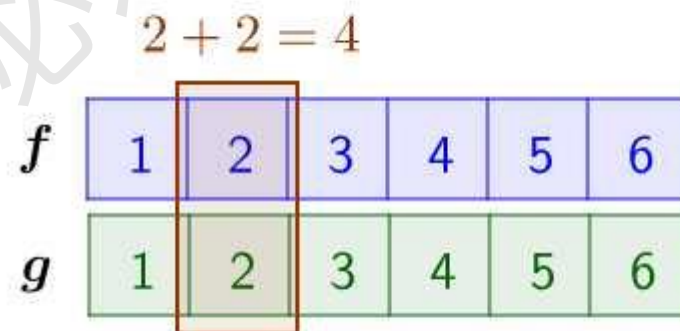
g 表示第二枚骰子

卷积神经网络 (CNN)

例子1：求扔两个色子结果之和为4的概率



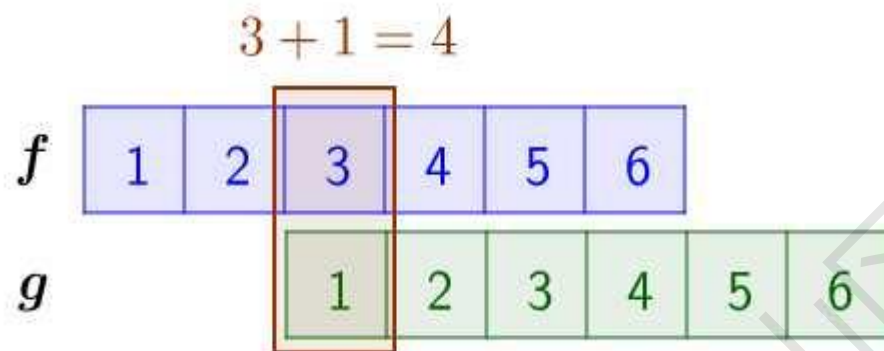
出现概率为： $f(1)g(3)$



出现概率为： $f(2)g(2)$

卷积神经网络 (CNN)

例子1: 求扔两个色子结果之和为4的概率



出现概率为: $f(3)g(1)$

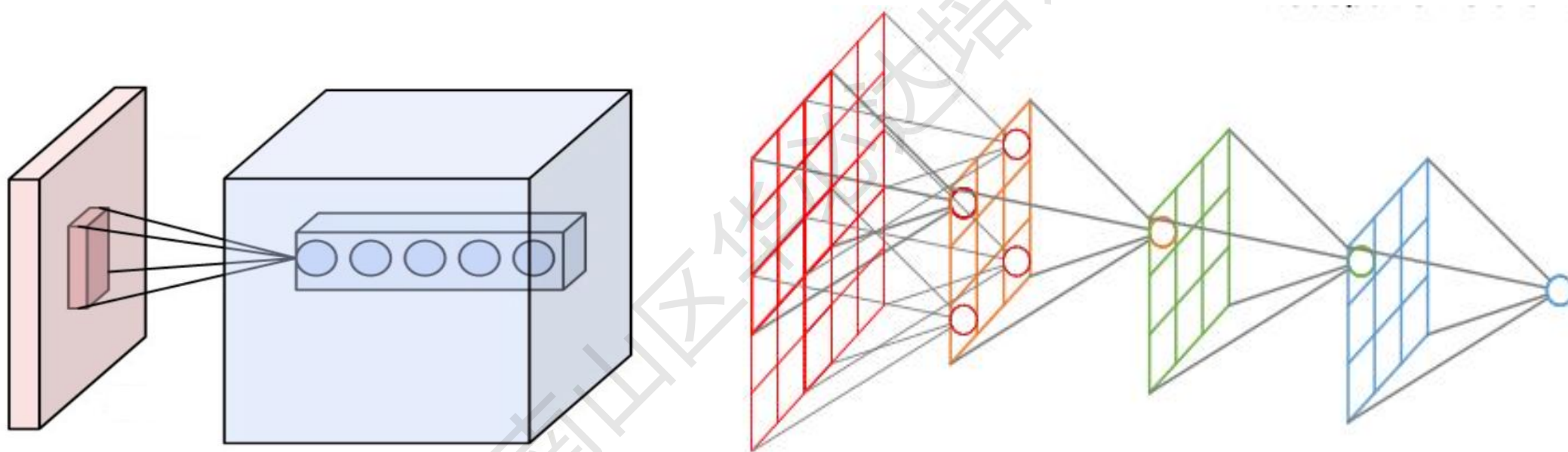
- 两枚骰子点数加起来为4的概率为:

$$f(1)g(3) + f(2)g(2) + f(3)g(1)$$

- 写成标准的形式:

$$(f * g)_4 = f_4 * g_4 = \sum_{m=1}^3 f_m * g(4-m)$$

卷积神经网络的感受野

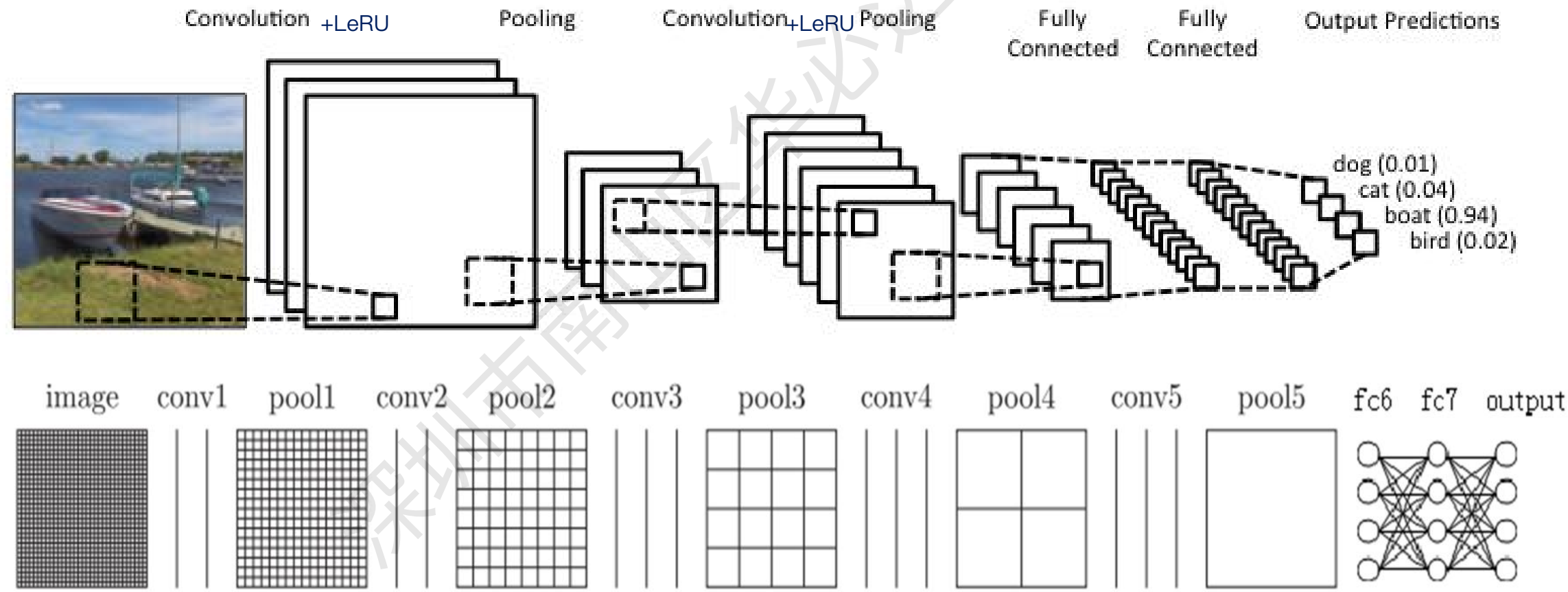


- **感受野**: 单个感觉神经元的感受野是感觉空间的特定区域 (如体表或视野), 在这个区域内, 刺激会改变神经元的放电

卷积神经网络的基本结构

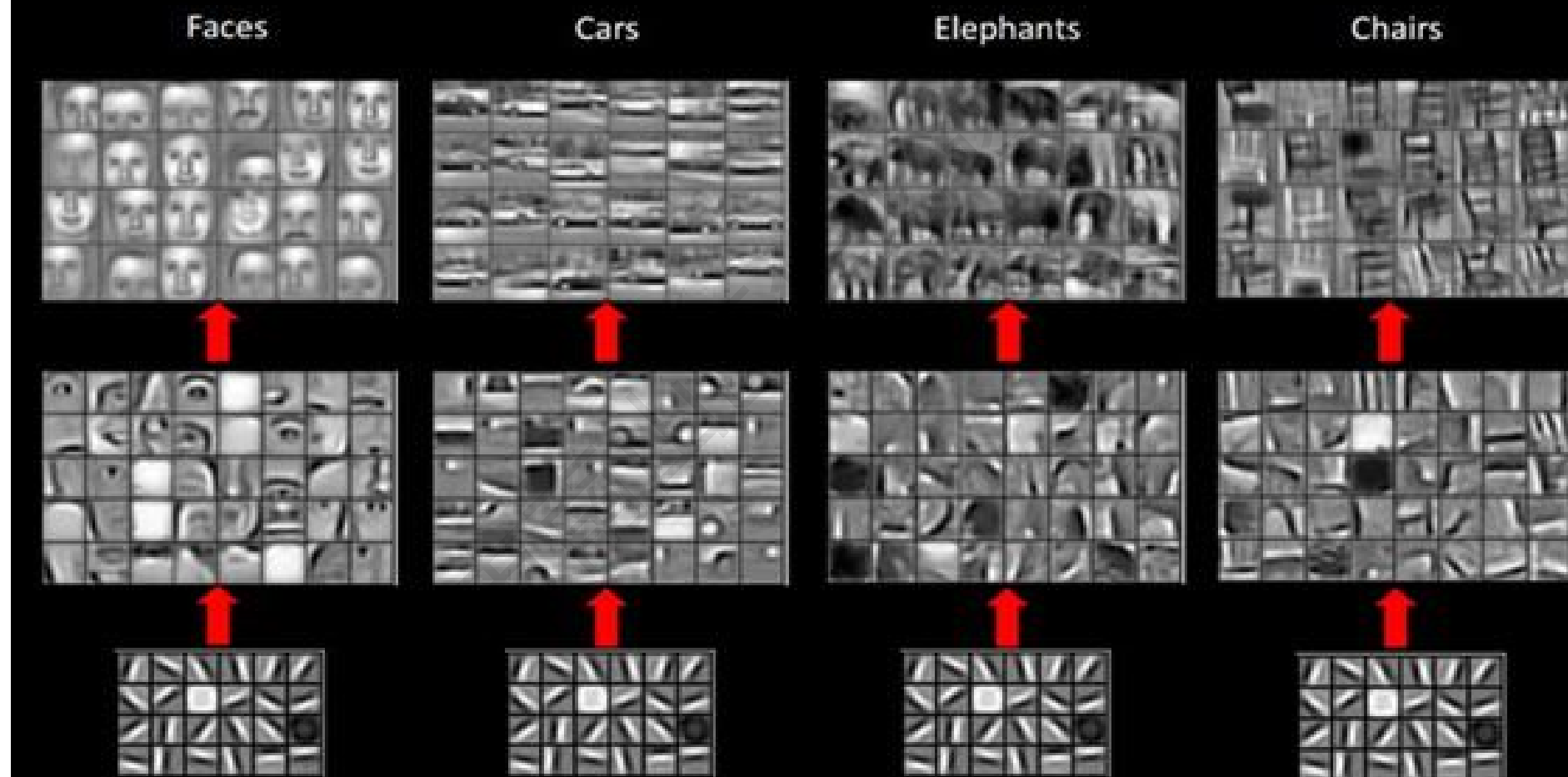
卷积神经网络的组成部分

- 卷积层+ReLU(Convolution)
- 池化层 (Pooling)
- 全连接层(Full-connected)



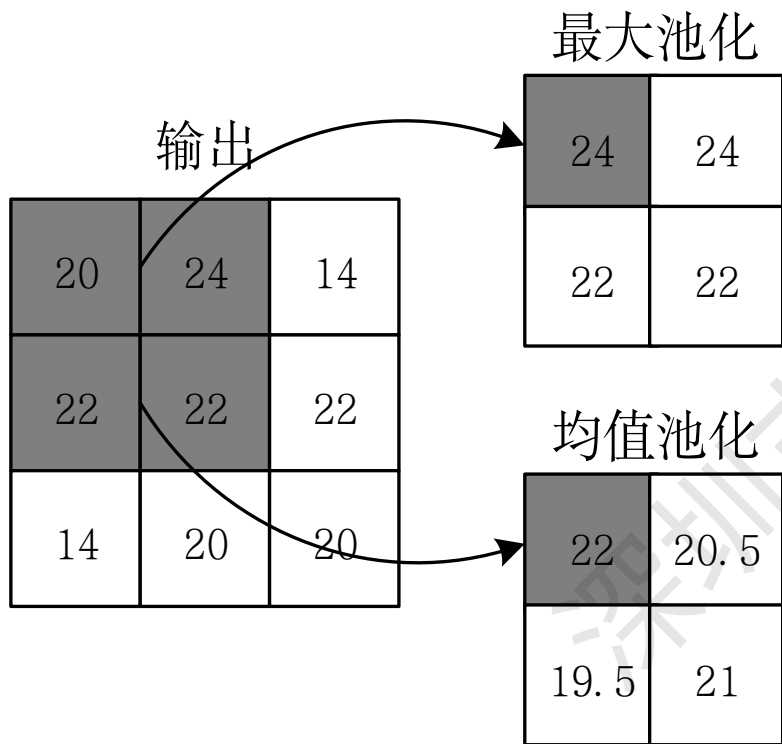
卷积层

Features learned from training on different object classes.



池化层

- 池化操作使用某位置相邻输出的总体统计特征作为该位置的输出，常用**最大池化 (max-pooling)** 和**均值池化 (average-pooling)**。
- 池化层不包含需要训练学习的参数，仅需指定池化操作的**核大小**、**操作步幅**以及**池化类型**

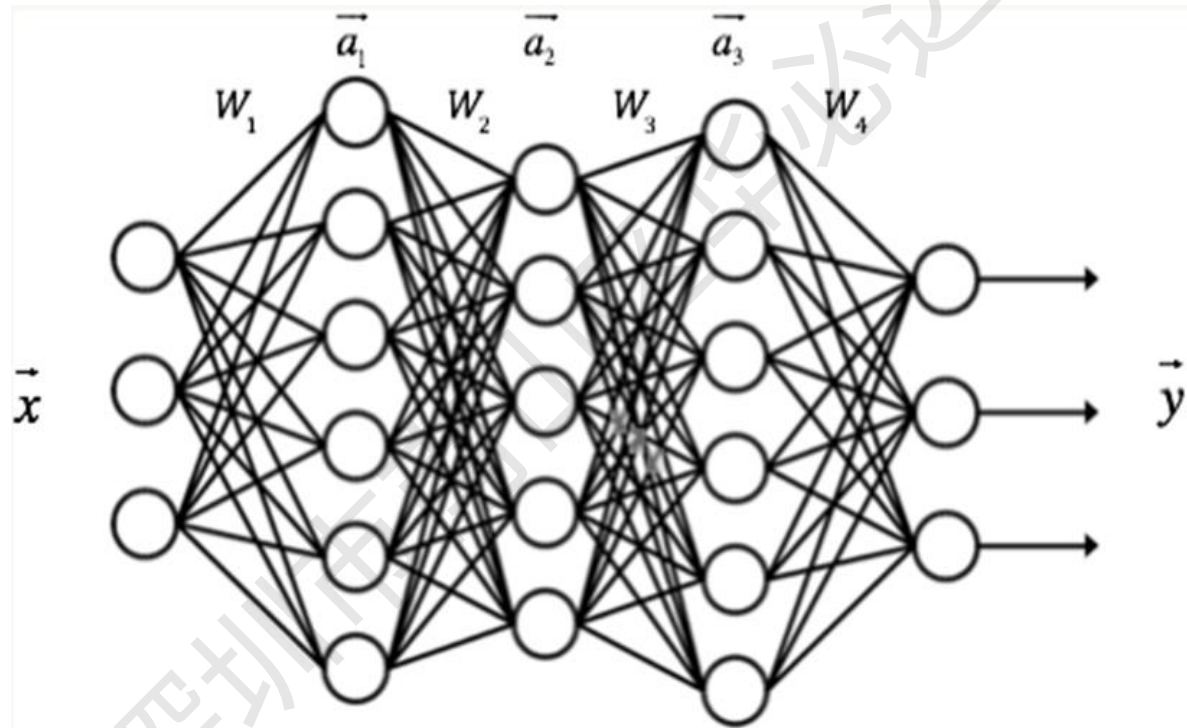


- 池化的作用

- 减少网络中的参数计算量，从而遏制过拟合
- 增强网络对输入图像中的小变形、扭曲、平移的**鲁棒性**(输入里的微小扭曲不会改变池化输出——因为我们在局部邻域已经取了最大值/平均值)。
- 帮助我们获得不因尺寸而改变的等效图片表征。这非常有用，因为这样我们就可以探测到图片里的物体，不管它在哪个位置。

全连接层

- 全连接层的目的是对卷积层和池化层输出的特征图（二维）进行降维。



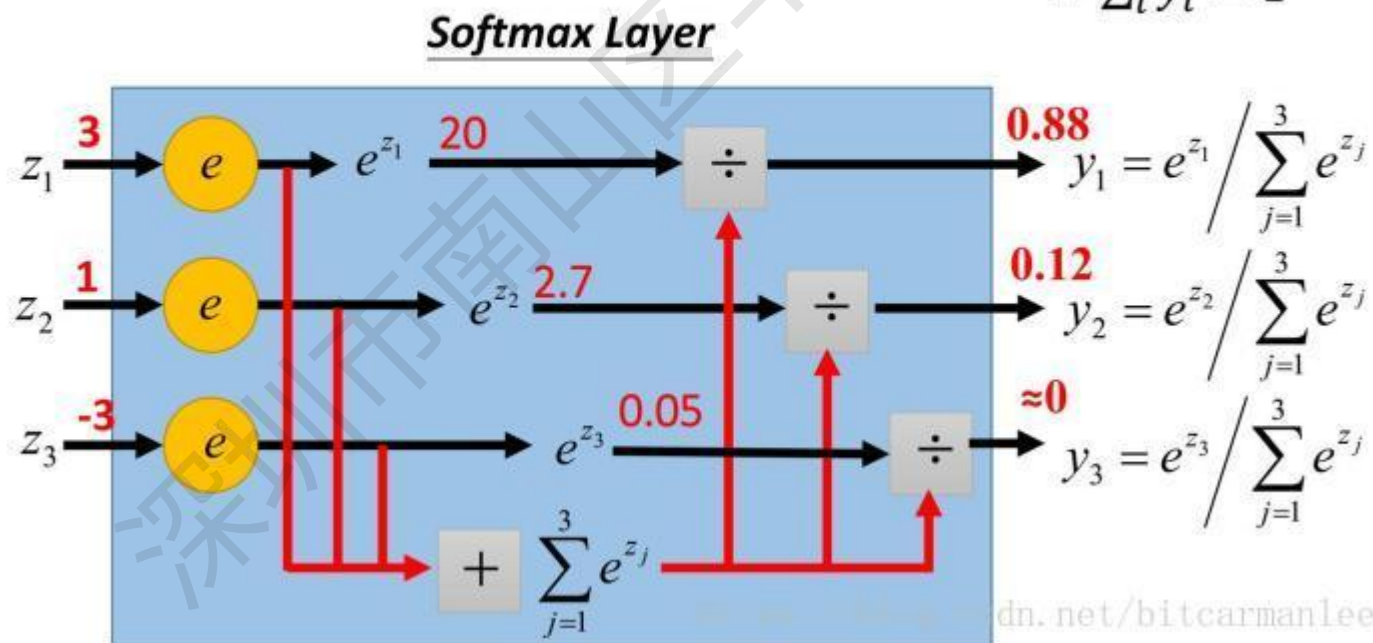
输出层

- 对于分类问题: 使用Softmax函数 $y_i = \frac{e^{z_i}}{\sum_{i=1}^n e^{z_i}}$
- 对于递归问题: 使用线性函数 $y_i = \sum_{m=1}^M w_{im} x_m$

- Softmax layer as the output layer

Probability:

- $1 > y_i > 0$
- $\sum_i y_i = 1$

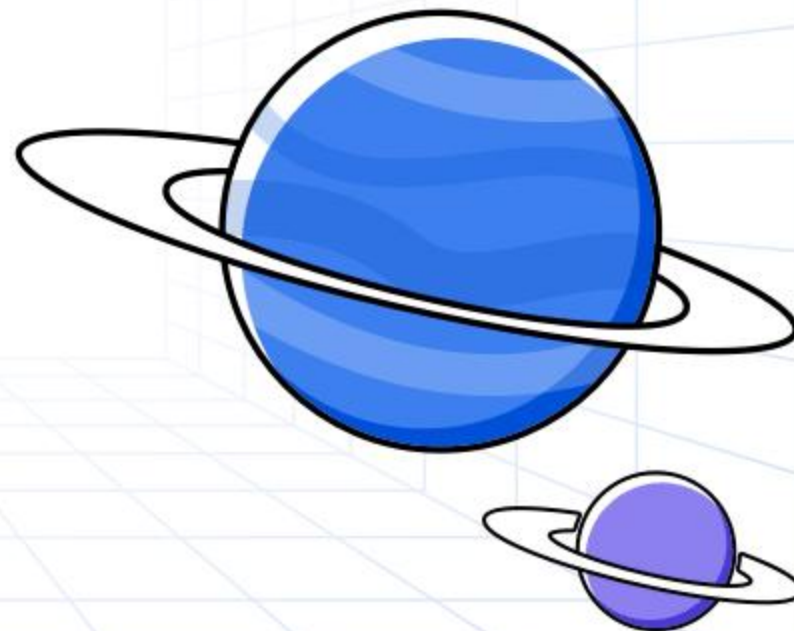


卷积神经网络的训练

- Step 1: 用随机数初始化所有的卷积核和参数/权重
- Step 2: 将训练图片作为输入，执行前向步骤（卷积，ReLU，池化以及全连接层的前向传播）并计算每个类别的对应输出概率。
- Step 3: 计算输出层的总误差：总误差 = $\frac{1}{2} \sum (\text{目标概率} - \text{输出概率})^2$
- Step 4: 反向传播算法计算误差相对于所有权重的梯度，并用梯度下降法更新所有的卷积核/权重和参数的值，以使输出误差最小化。
- 卷积核个数、卷积核尺寸、网络架构这些参数，是在Step 1之前就已经固定的，且不会在训练过程中改变——只有卷积核矩阵和神经元权重会更新。

05

算法分析



数据分析训练误差分析



● 数据分析训练误差

数据分析训练误差是指模型在训练数据上的误差，即模型预测结果与真实结果之间的偏差。训练误差分析可以帮助我们了解模型在训练数据上的表现，以及模型对于数据的拟合程度。

● 数据分析训练误差的来源

数据分析训练误差主要来源于模型的参数估计误差、数据噪声、数据维度过高、模型结构不足或过度拟合等因素。

● 数据分析训练误差的评估方法

数据分析训练误差的评估方法包括计算均方误差、均方根误差、平均绝对误差等指标，以及通过图形直观地展示预测结果与真实结果之间的偏差。

k交叉验证数据分析



k交叉验证

k交叉验证是一种常用的数据分割方法，用于评估模型的性能和稳定性。在k交叉验证中，数据被分成k个不同的子集，每个子集都有相同的大小和分布。

k交叉验证的优点

k交叉验证可以帮助我们更准确地评估模型的性能，因为数据被分成多个不同的子集，可以使得评估结果更具代表性。同时，k交叉验证也可以帮助我们了解模型的稳定性，即模型在不同数据子集上的表现是否稳定。

k交叉验证的注意事项

在k交叉验证中，需要选择合适的k值，一般推荐k=5或k=10。同时，还需要注意数据分割的随机性和均衡性，以确保评估结果的准确性和可靠性。

数据分析协方差与标准误差



● 数据分析协方差

数据分析协方差是衡量两个变量之间相关性的统计量。如果两个变量同时向同一方向变化，则协方差为正数；如果两个变量向相反方向变化，则协方差为负数。

● 数据分析标准误差

数据分析标准误差是描述数据分布散度的统计量，它表示数据的波动范围。标准误差越小，说明数据越集中、越稳定。

● 数据分析协方差与标准误差的关系

数据分析协方差和标准误差是衡量数据特性的两个不同统计量，但它们之间有一定的联系。例如，如果两个变量之间的协方差为正数，则说明它们之间是正相关关系，此时标准误差可能会受到一定的影响。

算法度量外部指标



Rand统计量

Rand统计量是一个随机变量，它是基于样本数据计算出的用于推断总体的某些量的统计量。这个概念涉及到统计学中的抽样分布，即由样本数据计算出的统计量所形成的概率分布。

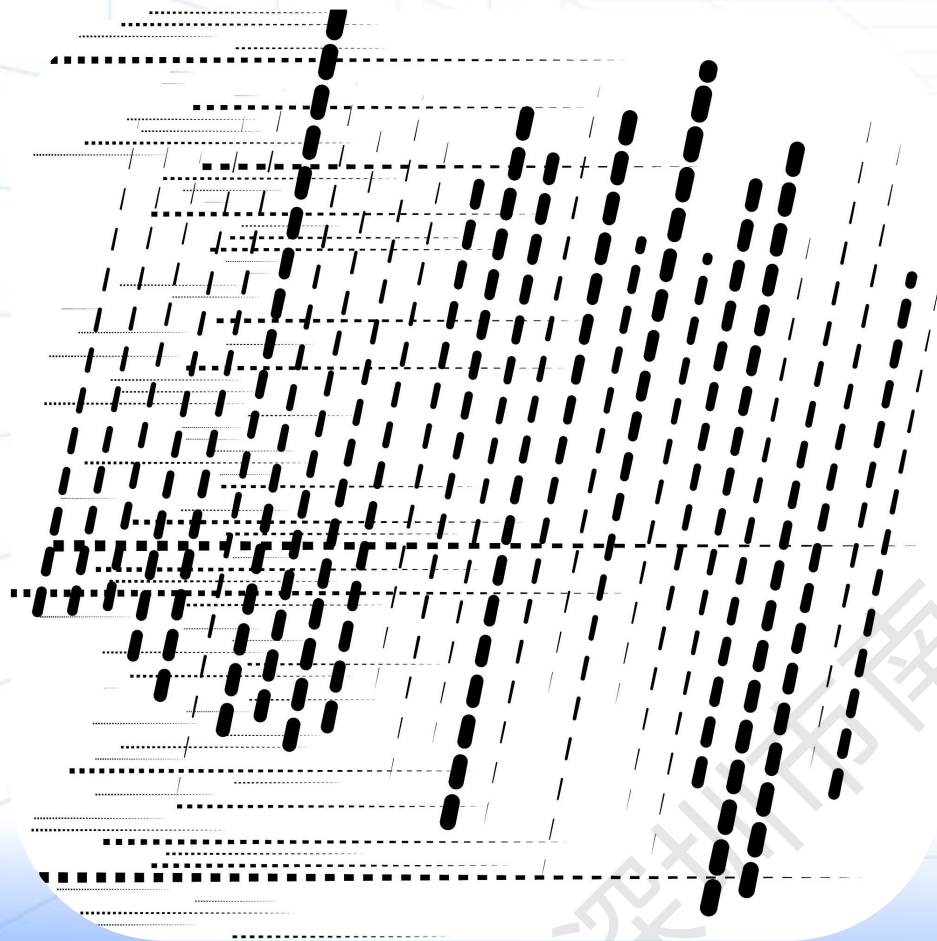
Jaccard系数

Jaccard系数，也称为Jaccard相似系数，是一种用于比较有限样本集之间相似性与差异性的度量方法。它通过计算两个集合的交集大小与并集大小的比值来度量它们的相似性。

F1 分数

F1 分数是精确度和召回率的调和平均值，它提供了一个统一的评价指标。计算 F1 分数的方法是， $2 * \text{精确度} * \text{召回率} / (\text{精确度} + \text{召回率})$ 。

算法复杂度分析



● 时间复杂度

时间复杂度是指算法执行时间随数据规模的变化情况。一般来说，时间复杂度越低，算法的效率就越高。

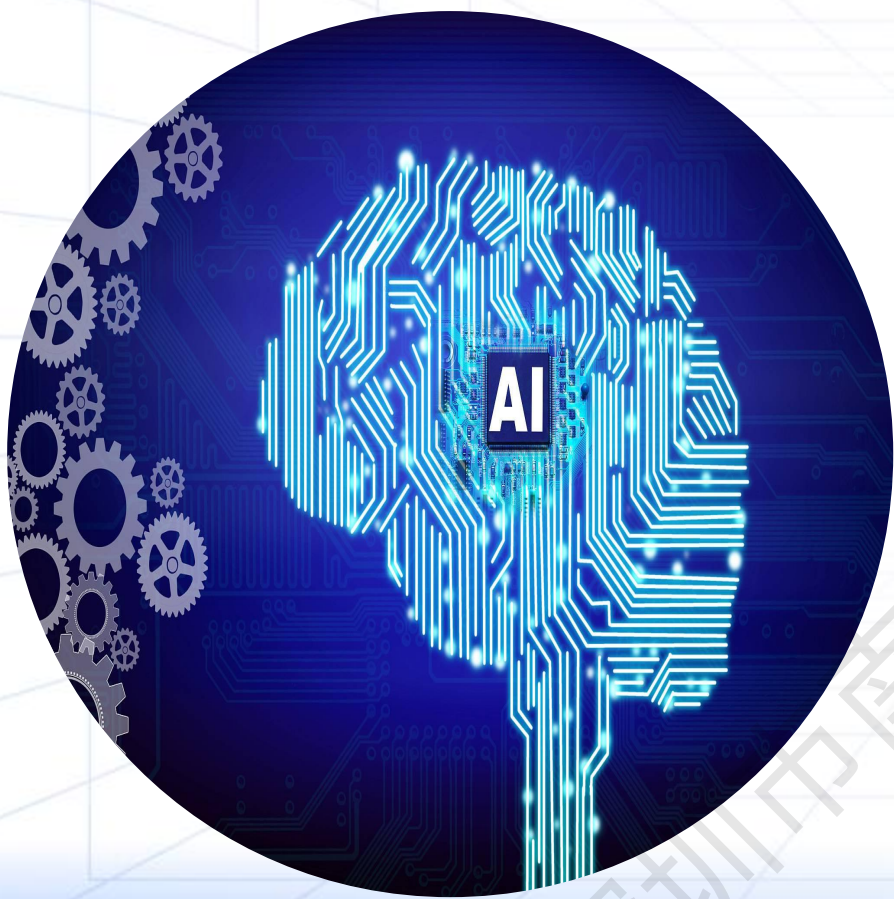
● 空间复杂度

空间复杂度是指算法所需存储空间随数据规模的变化情况。一般来说，空间复杂度越低，算法的存储效率就越高。

● 稳定性分析

稳定性分析是指算法在不同输入数据下的表现情况。一般来说，稳定性越高的算法越能够应对不同的输入数据。

算法度量内部指标，以聚类为例



紧密性

紧密性度量了同一个簇中样本之间的相似程度，即簇内样本越相似，紧密性越高。常见的紧密性计算方法包括欧氏距离、曼哈顿距离等。一般来说，紧密性越高，表示聚类结果越好。

分离度

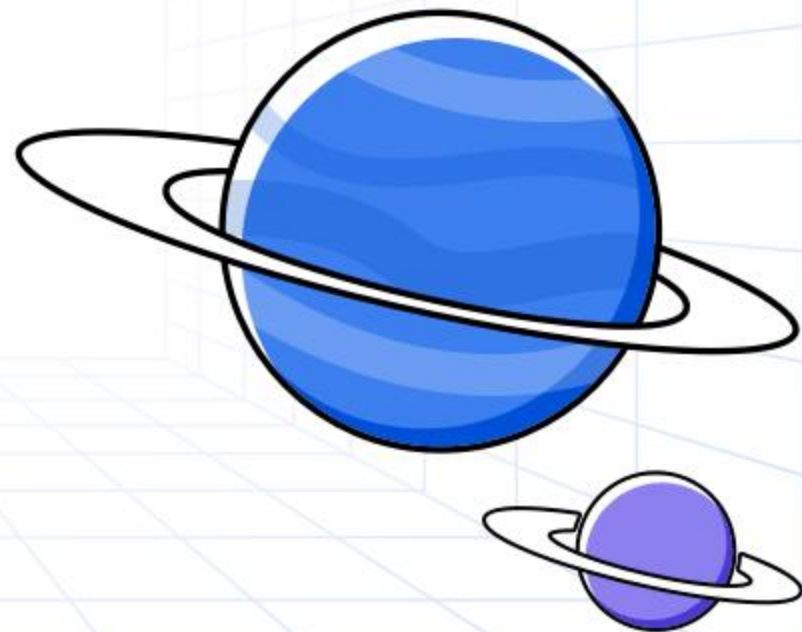
分离度度量了不同簇之间样本的差异程度，即簇间样本越不相似，分离度越高。常见的分离度计算方法包括最近邻距离、最远邻距离等。一般来说，分离度越高，表示聚类结果越好。

轮廓系数

轮廓系数综合考虑了紧密性和分离度，是一种常用的内部评价指标。对于每个样本，轮廓系数计算了它与所在簇内其他样本的平均距离（紧密性）和与最近簇中样本的平均距离（分离度），并将两者进行比较。轮廓系数的取值范围为 $[-1, 1]$ ，越接近1表示聚类结果越好。

06

算法可视化分析



算法可视化分析方法



图表和图形

常见的算法可视化图表包括折线图、柱状图、饼状图、散点图、气泡图等，这些图表可以帮助我们直观地理解算法的性能和趋势。



可视化工具

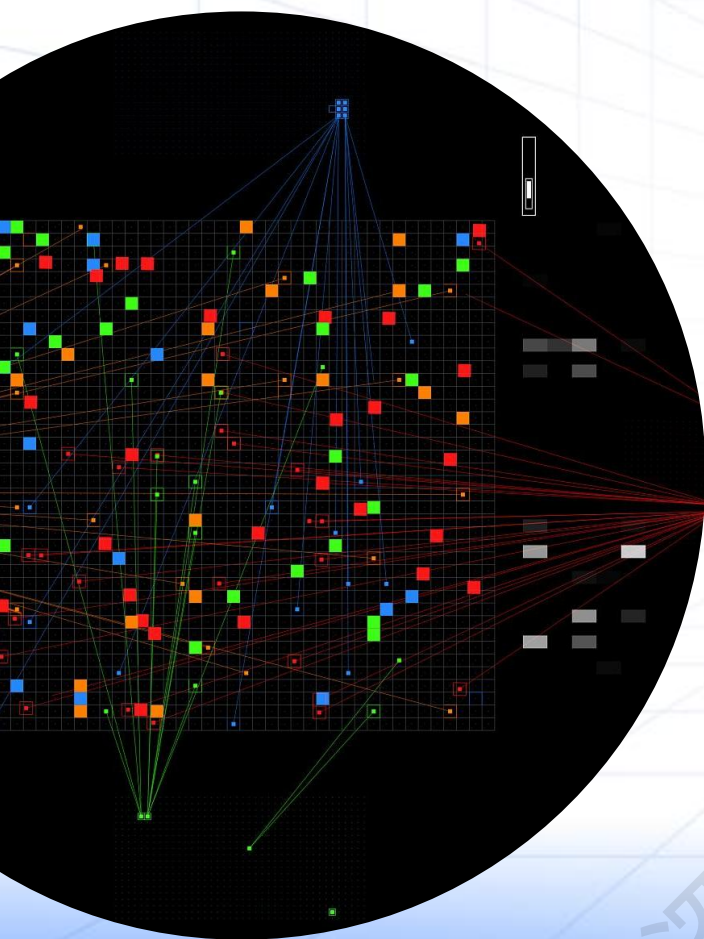
常见的算法可视化工具包括 Matplotlib、Seaborn、Shiny 等；
常见的工具：Excel、WPS、Tableau、Python、Raw 等。



动画演示

我们可以使用算法可视化工具制作动画演示，以帮助人们更好地理解算法的工作原理和性能变化。

算法可视化分析常用工具



Matplotlib

Matplotlib 是一个流行的Python绘图库，可以制作高质量的图表和图像，帮助人们直观地理解数据。



Seaborn

Seaborn 是一个基于 Matplotlib 的Python数据可视化库，它提供了更多高级的数据可视化功能。



Shiny

Shiny 是一个用于制作交互式网页应用程序的R包，可以帮助人们将数据以多种形式进行可视化展示。



ggplot

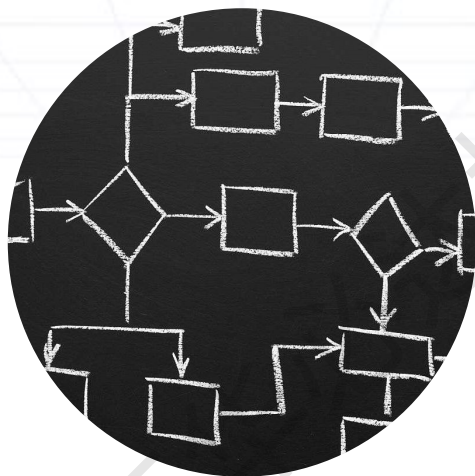
ggplot2是一款强大的图形可视化R包，其作图方式易于理解，且生成的图形精美，定制化程度也很高，应该是R里面最流行的可视化工具。可以将多个图层叠加。

算法可视化图表介绍



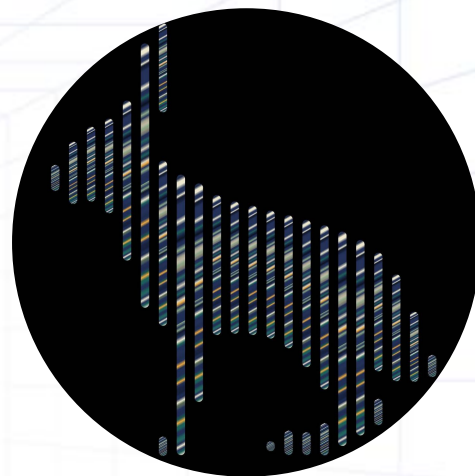
流程图

流程图是一种常用的算法可视化图表，它可以通过直观的图形表示算法的执行过程，帮助人们更好地理解算法的逻辑。



数据结构图

数据结构图是一种用于表示数据结构的图表，它可以帮助人们更好地理解数据元素之间的关系和特性。

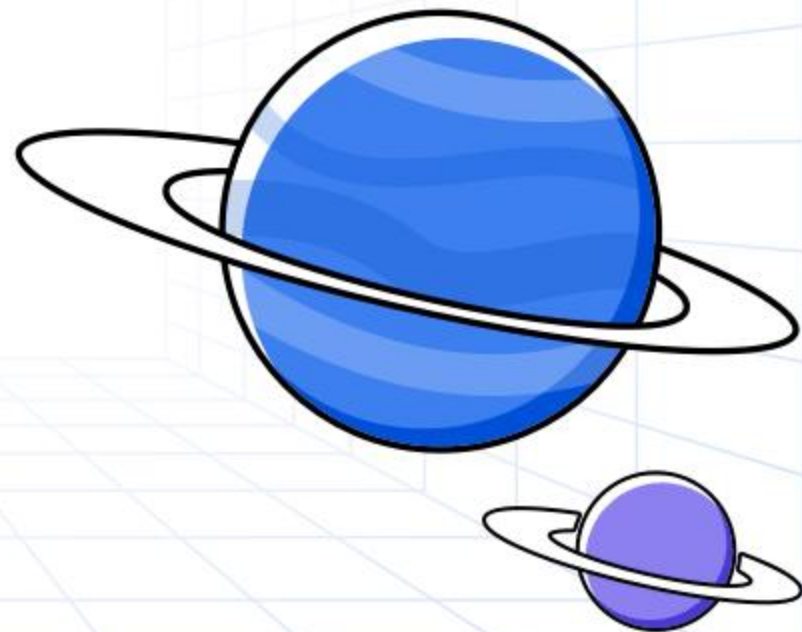


算法性能图

算法性能图是一种用于评估算法性能的图表，它可以通过图表的形式展示算法在各种性能指标上的表现。

07

训练数据收集与算法模型训练



训练数据收集方法

人工收集数据

通过人工方式收集数据，例如通过问卷调查、访谈、文献资料等多种方式，收集相关数据并整理成训练数据集。

爬虫技术

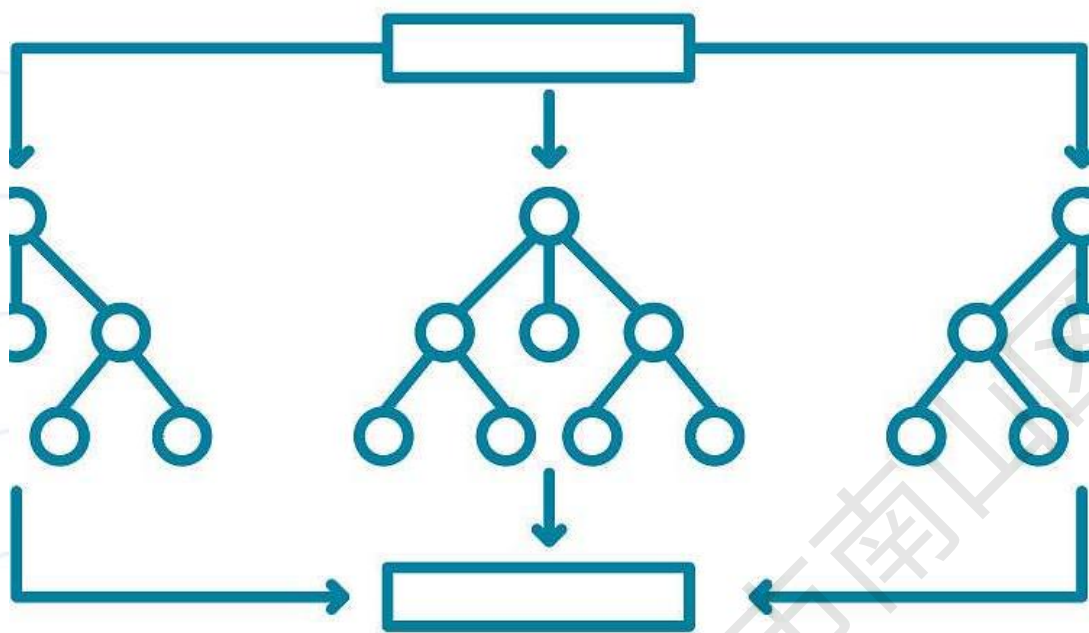
利用爬虫技术从互联网爬取相关数据，例如使用Python的BeautifulSoup和Scrapy等工具，可以方便地爬取网页数据并整理成训练数据集。

数据库查询

通过数据库查询的方式收集数据，例如使用SQL语言查询数据库中的数据，可以按照需求提取相关数据并整理成训练数据集。



算法模型训练过程



数据预处理

对训练数据进行清洗、转换和加载等操作，以便适应算法模型的输入需求。

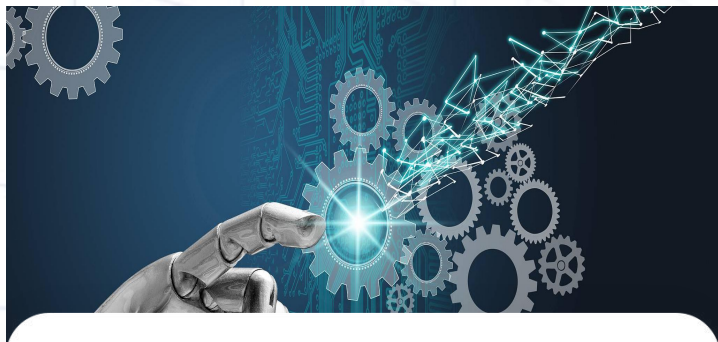
模型选择

根据需求选择合适的算法模型，例如K-近邻、决策树、神经网络等。

参数调整

根据需求调整算法模型的参数，例如K-近邻的K值、决策树的分割点等。

留出法：留出法是一种常用的分类效果评价方法，它通过将数据集分成训练集和测试集来评估算法的性能。



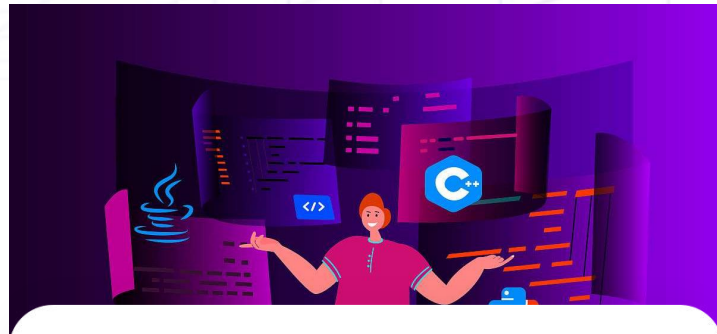
留出数据集

将一部分数据作为留出数据集，用于评估算法模型的性能。



训练模型

在留出数据集上训练算法模型，并调整参数使模型达到最佳状态。



评估性能

在留出数据集上评估算法模型的性能，例如计算准确率、召回率等指标。

交叉验证：交叉验证法是一种更可靠的分类效果评价方法，它通过多次划分数据集来评估算法的稳定性。

评估性能

在1份数据上评估算法模型的性能，
计算准确率、召回率等指标。

重复实验

重复上述实验10次，每次选择不同的
1份数据用于评估性能。

平均性能

将10次实验的性能指标平均值作为算
法模型的性能指标。

分割数据

将训练数据分成10份，其中9份用于
训练模型，1份用于评估性能。

训练模型

在9份数据上训练算法模型，并调整
参数使模型达到最佳状态。



算法过拟合问题及其解决方式



● 过拟合问题定义

算法模型在训练数据上表现良好，但在测试数据上表现较差。

● 过拟合原因

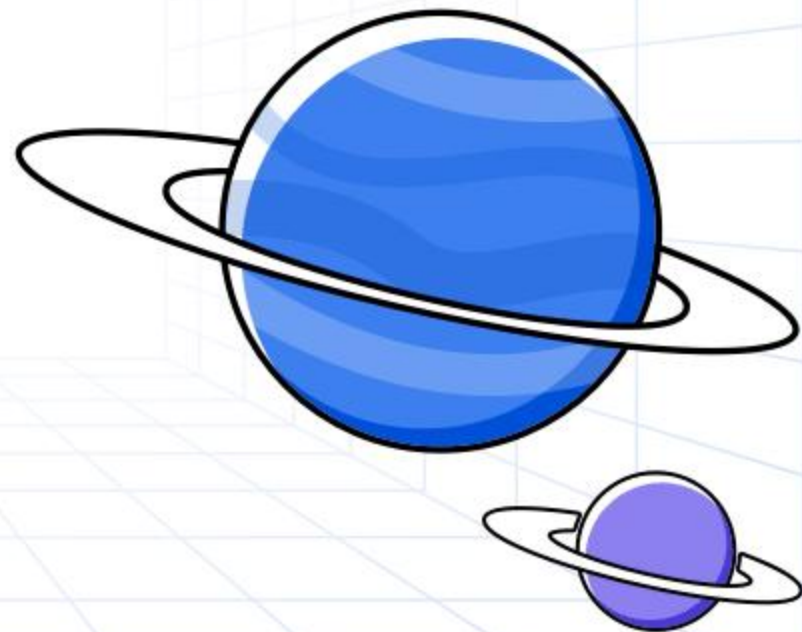
可能是由于算法模型过于复杂，或者训练数据不足等原因引起的。

● 解决过拟合问题的方式

增加训练数据，使模型更好地学习到数据的分布规律。降低模型复杂度，使模型更容易泛化到新数据上。使用正则化，限制模型的复杂度并使模型更加稳健。

07

神经网络算法效果 评价

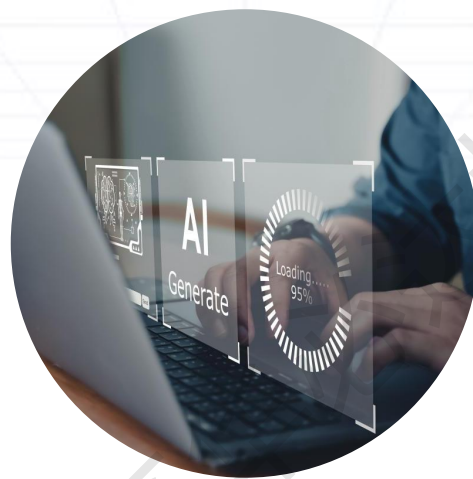


算法学习率调整策略



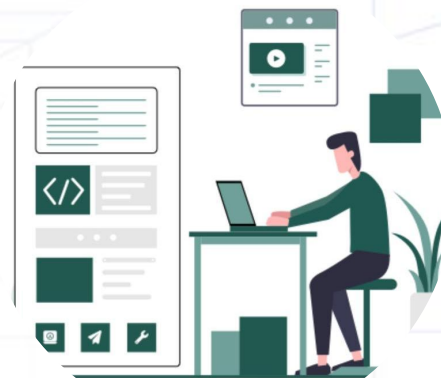
初始学习率设置

初始学习率应该根据问题的复杂性和数据的大小来设置。一般来说，初始学习率应该较小，以避免算法在训练过程中陷入局部最优解。



学习率调整时机

学习率应该根据训练过程进行调整。一般来说，当算法在训练过程中达到一定的准确率或损失函数值不再显著下降时，应该减少学习率。



学习率调整策略

有多种学习率调整策略，如指数衰减、多项式衰减、周期性学习和Adam等。这些策略可以根据具体的训练过程和需求进行选择。

算法测试参数优化方法

网格搜索

网格搜索是一种通过遍历多个参数值来找到最优参数的方法。具体做法是，将每个参数的值划分成多个不同的段，然后分别测试这些组合下的模型性能。

遗传算法

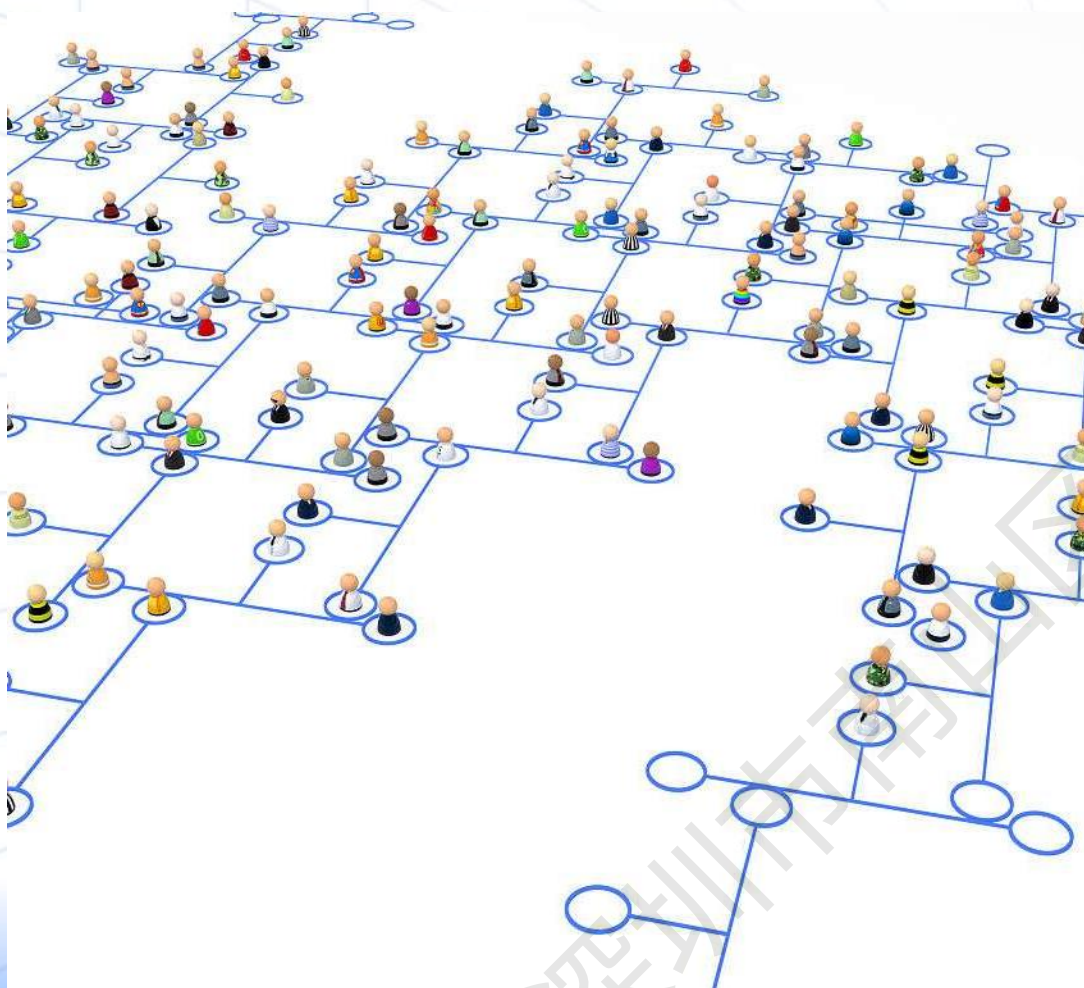
遗传算法是一种优化算法，它借鉴了生物进化原理，通过模拟自然界中生物的进化过程来寻找最优参数。

随机搜索

随机搜索是一种通过随机选取参数值来测试模型性能的方法。虽然这种方法比网格搜索和遗传算法耗时更长，但可能在更短的时间内找到最优参数。



混合精度训练



使用GPU加速

利用GPU加速混合精度训练可以显著提高训练速度和质量。具体做法是，将部分计算过程放在GPU上进行，而其余部分则放在CPU上进行。

选择合适的数据类型

训练神经网络模型的时候默认使用的数据类型为单精度FP32。近年来，为了加快训练时间、减少网络训练时候所占用的内存，并且保存训练出来的模型精度持平的条件下，业界提出越来越多的混合精度训练的方法。这里的混合精度训练是指在训练的过程中，同时使用单精度（FP32）和半精度（FP16）。

注意数值稳定性

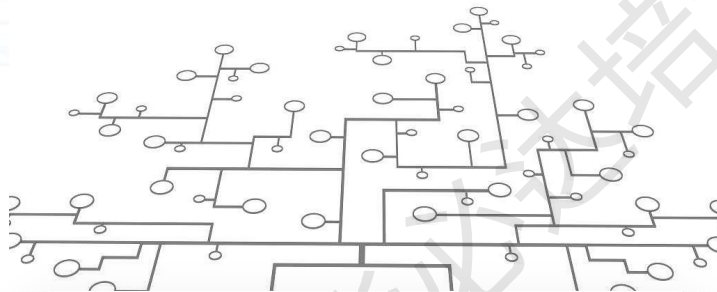
在混合精度训练中，要注意数值稳定性问题。具体做法是，使用一些数值稳定的函数和算法来实现计算过程。混合精度训练过程中可以引入权重备份（Weight Backup）、损失放大（Loss Scaling）、精度累加（Precision Accumulated）三种相关的技术。

梯度下降算法：是一种用于寻找函数最小值的优化算法



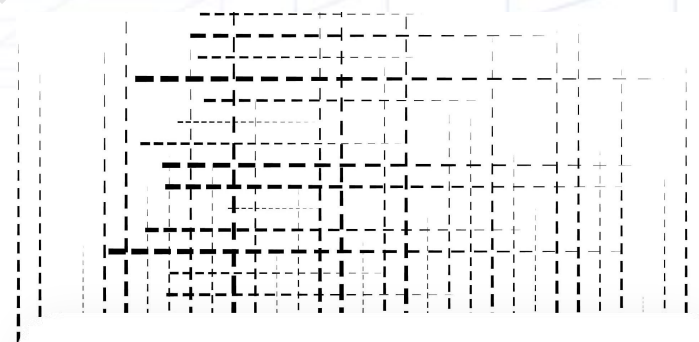
选择合适的步长

在梯度下降算法中，选择合适的步长非常重要。一般来说，应该根据问题的特性和数据的规模来设定步长。



设定合适的迭代次数

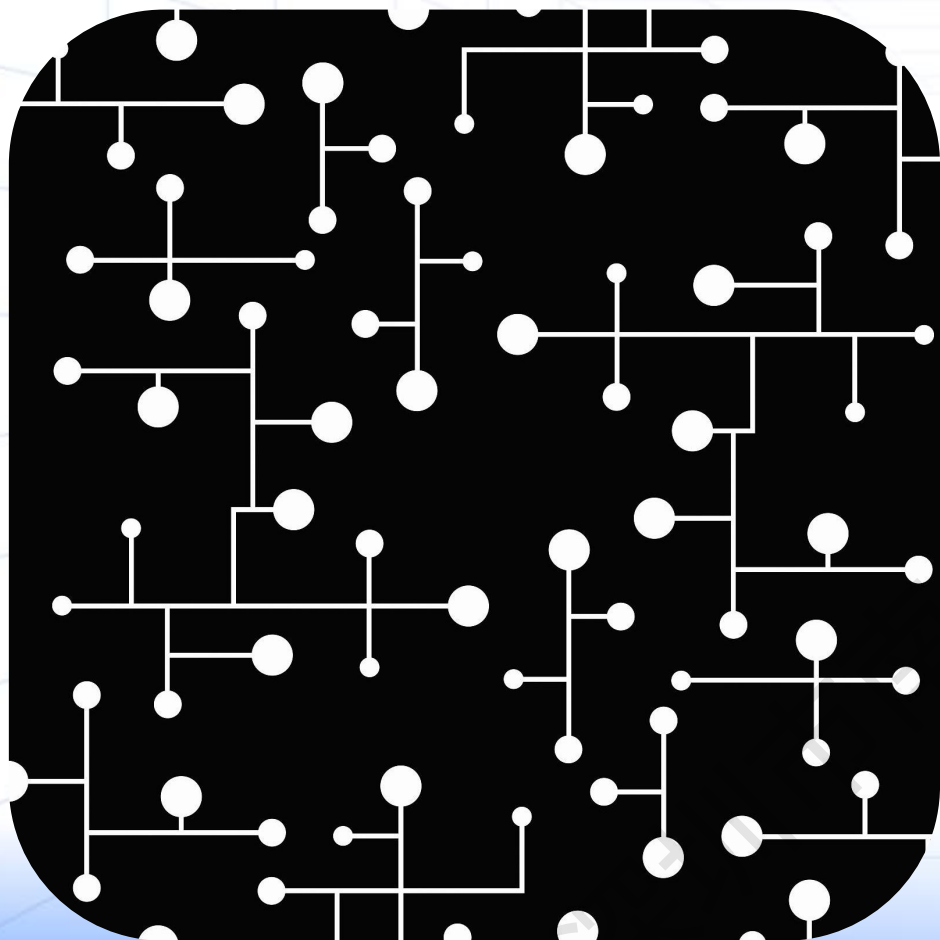
在梯度下降算法中，设定合适的迭代次数也非常重要。一般来说，应该根据问题的复杂性和数据的规模来设定迭代次数。



注意梯度消失问题

在梯度下降算法中，要注意梯度消失问题。具体做法是，使用一些防止梯度消失的技术或者更改梯度计算的方式。

遗传算法（Genetic Algorithm, GA）：是一种进化算法，其基本原理是仿效生物界中的“物竞天择、适者生存”的演化法则



● 选择合适的种群大小

在遗传算法中，选择合适的种群大小非常重要。一般来说，种群大小应该根据问题的复杂性和数据的规模来设定。

● 设定合适的变异概率

在遗传算法中，设定合适的变异概率也非常重要。一般来说，变异概率应该根据问题的特性和数据的规模来设定。

● 注意避免早熟收敛

在遗传算法中，要注意避免早熟收敛问题。具体做法是，使用一些避免早熟收敛的技术或者更改遗传算法的实现方式。

模拟退火算法

1

设定合适的初始温度

在模拟退火算法中，设定合适的初始温度非常重要。一般来说，初始温度应该根据问题的复杂性和数据的规模来设定。

2

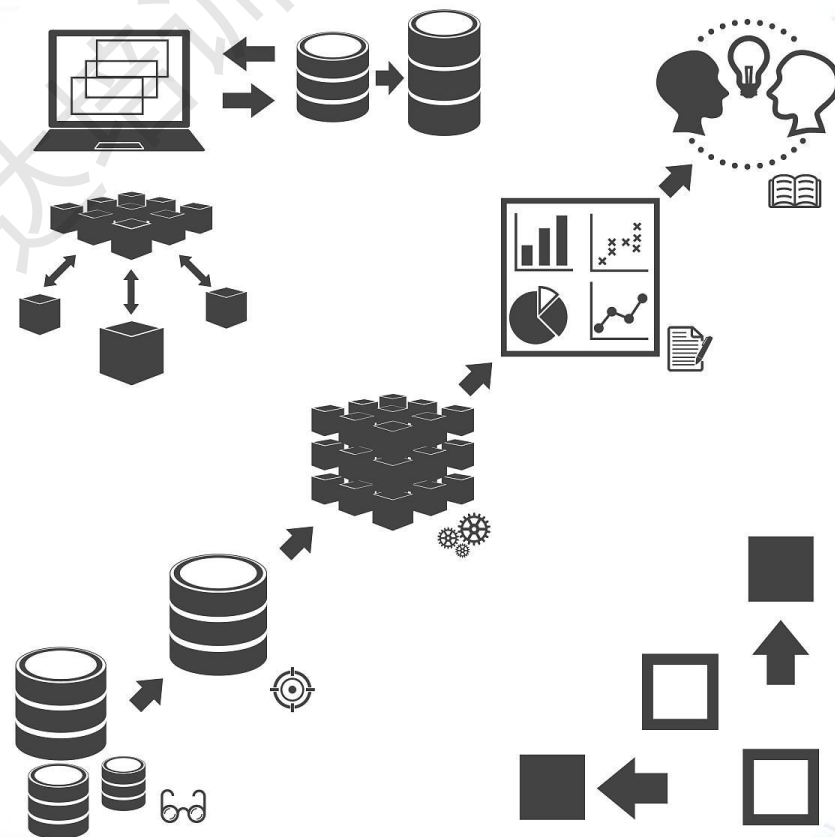
注意控制降温速度

在模拟退火算法中，注意控制降温速度也非常重要。一般来说，降温速度应该根据问题的特性和数据的规模来调整。

3

注意选择邻域结构

在模拟退火算法中，注意选择邻域结构也非常重要。一般来说，邻域结构应该根据问题的特性和数据的规模来选择。



遗传算法

1. 遗传算法的本质也是通过一定地策略去寻找最优解。
2. 遗传算法则是运用了生物学自然选择的原理
3. 遗传算法是一种随机全局搜索优化，通过模拟自然选择和遗传中的复制、交叉、变异来实现“优胜劣汰”，最终找到最优解。

模拟退火算法

1. 遗传算法的本质也是通过一定地策略去寻找最优解。
2. 模拟退火运用了动力学原理

蚁群算法：属于一种基于群体智能的优化算法，通过模拟蚂蚁在路径上释放信息素的行为来解决复杂的优化问题

选择合适的蚂蚁数量

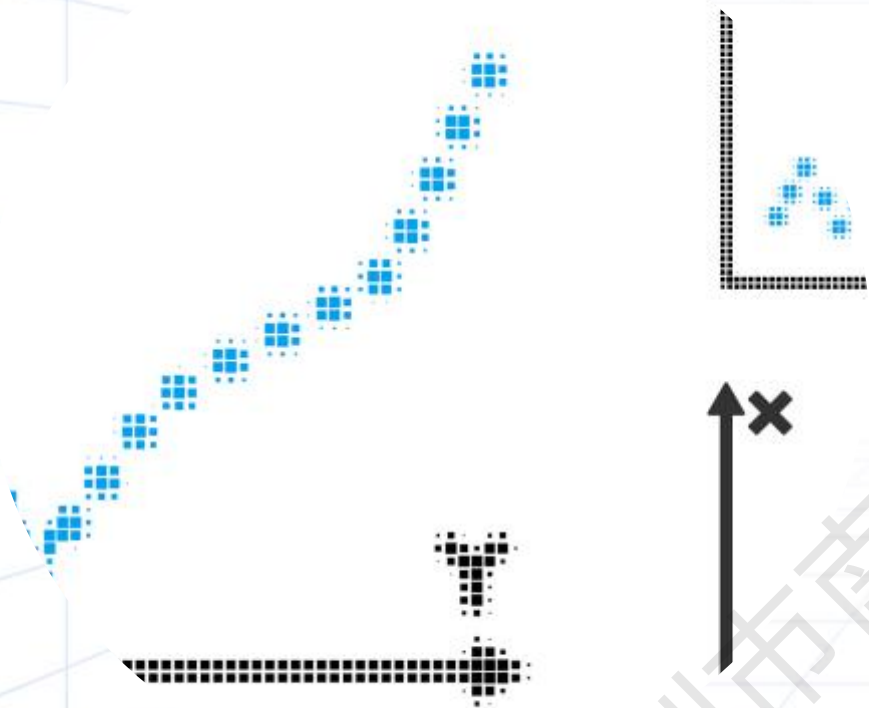
在蚁群算法中，选择合适的蚂蚁数量非常重要。一般来说，蚂蚁数量应该根据问题的复杂性和数据的规模来设定。

设定合适的信息素挥发速度

在蚁群算法中，设定合适的信息素挥发速度也非常重要。一般来说，信息素挥发速度应该根据问题的特性和数据的规模来调整。

注意避免局部最优解

在蚁群算法中，要注意避免局部最优解问题。具体做法是，使用一些避免局部最优解的技术或者更改蚁群算法的实现方式。



随机搜索：是一种超参数调优的方法，用于在机器学习模型中找到最佳的超参数组合。

确定搜索范围

在随机搜索中，确定搜索范围非常重要。一般来说，搜索范围应该根据问题的复杂性和数据的规模来确定。

选择合适的搜索步长

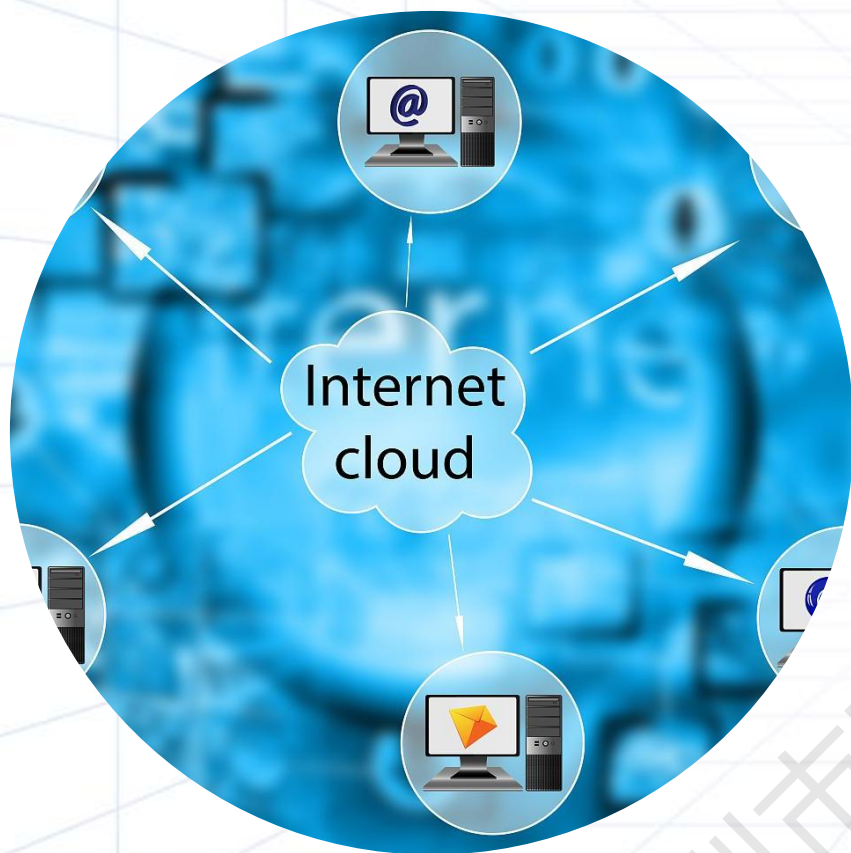
在随机搜索中，选择合适的搜索步长也非常重要。一般来说，搜索步长应该根据问题的特性和数据的规模来确定。

注意搜索效率

在随机搜索中，要注意搜索效率问题。具体做法是，使用一些提高搜索效率的技术或者更改随机搜索的实现方式。

相对于传统的网格搜索，随机搜索的主要特点是不按照预定的网格顺序搜索超参数空间，而是随机选择超参数的组合进行评估。

测试报告



测试报告目标

明确测试报告的目标和范围，确保报告具有可读性和可操作性。

测试报告内容

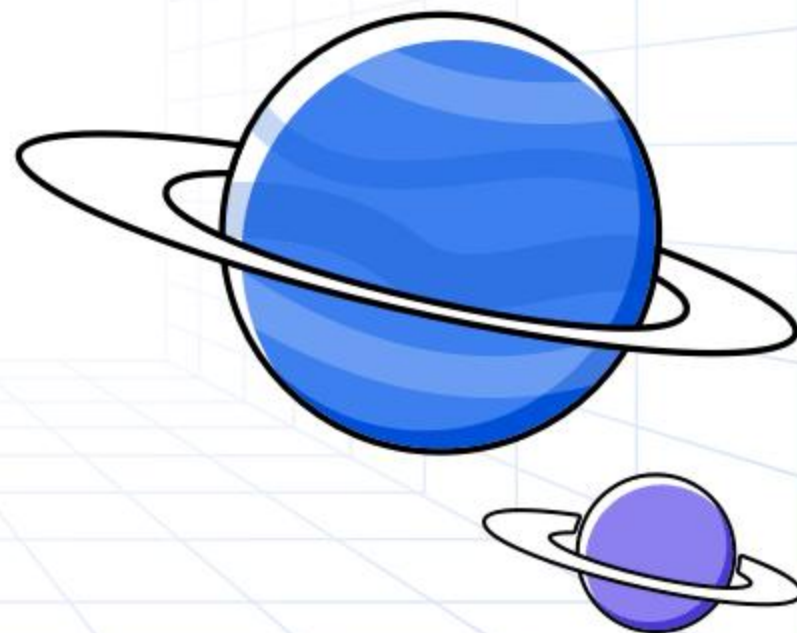
详细描述测试的过程、结果和分析，包括测试的目标、范围、方法和技术要求。

测试报告审核

对测试报告进行审核和评估，确保报告的准确性和完整性。

09

算法检测与分类效果评价



算法测试分类效果评价指标

True Positive(真正, TP):

样本的真实标签为正类, 算法预测也为正类数

True Negative(真负, TN):

样本的真实标签为负类, 算法预测也为负类数

False Positive(假正, FP):

样本的真实标签为负类, 算法预测为正类数 →

误报 误伤 误召回

False Negative(假负, FN):

样本的真实标签为正类, 算法预测为负类数 →

漏报

准确率

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

准确率是分类效果的重要指标, 它表示算法正确分类的样本数占总样本数的比例。

精度

$$\text{Precision} = \frac{TP}{TP + FP}$$

精度是另一个常用的分类效果评价指标, 它表示算法正确分类的样本数占有所有被预测为正的样本数的比例。

召回率

$$\text{Sensitivity} = \text{Recall} = \text{TPR} = \frac{TP}{TP + FN}$$

召回率也是分类效果的重要指标, 它表示算法正确分类的样本数占有所有实际为正的样本数的比例。

算法模型测试中的常见问题及解决策略

过拟合问题

过拟合问题是指算法模型在训练数据上表现良好，但在测试数据上表现较差的情况。解决策略包括增加训练数据、减少模型复杂度、使用正则化等。

欠拟合问题

欠拟合问题是指算法模型在训练数据上表现较差的情况。解决策略包括增加模型复杂度、减少训练数据、调整学习率等。

学习率调整问题

学习率调整问题是指在学习过程中，学习率的调整不当导致算法性能下降的情况。解决策略包括选择合适的学习率、使用学习率衰减等。



◆ THANKS ◆
感谢您的观看

